

CSA4008 - APPLIED MACHINE LEARNING



Presented By

Dr Jasmine Selvakumari Jeya I

Senior Associate Professor

School of Computing Science and Engineering

VIT Bhopal University

Bhopal - Indore Highway, Madhya Pradesh – 466144

jasmineselvakumarijeya@vitbhopal.ac.in

Learning a Class from Examples, Linear, Non-linear, Multi-class and Multi- label classification

- Learning a class from examples refers *to the process where a machine learning model learns to categorize data* (like images, text, or numbers) into a **specific class** based on **training examples** *labeled* as either **positive** or **negative**.
- **Procedure:**
 - **Collect Training Data:** Labeled examples are gathered.
 - **Select Features:** Decide which features of the data will be used (e.g., height, weight).
 - **Train the Model:** Use an algorithm (e.g., decision tree, SVM) to find patterns.
 - **Evaluate the Model:** Test how well it predicts new/unseen data.
 - **Predict:** Use the trained model to classify new inputs.

Example:

- Suppose we want to classify cars as family cars or not based on:
 - Price (x1)
 - Engine power (x2)

Price (x1)	Engine Power (x2)	Label
5 lakhs	80 HP	+
10 lakhs	120 HP	-
6 lakhs	75 HP	+
12 lakhs	150 HP	-

- Here, + means "Family car" and – means "Not a family car". The algorithm will learn the boundary (say, low price and low power → family car).

2. Linear Classification

In **linear classification**, the model tries to separate classes using a **straight line** (in 2D), a **plane** (in 3D), or a **hyperplane** (higher dimensions).

- **Procedure:**

- **Plot data points.**
- **Train a linear classifier** like Logistic Regression or SVM
- **Find the linear boundary (decision surface)** that best separates the classes.
- **Use this boundary** to classify new points

Example:

- We want to classify whether a student **passes or fails** based on:
- Study Hours (x)
- Sleep Hours (y)

x (Study)	y (Sleep)	Label
2	8	Fail
5	6	Pass
1	9	Fail
6	5	Pass

- A linear classifier may learn a boundary like:
- **if (Study Hours > 4), then → Pass**

3. Non-Linear Classification

Non-linear classification is used when classes can't be separated by a straight line. The decision boundary is curved or more complex.

- Procedure:**

- Plot Data:** Notice that a straight line can't separate the data
- Use a non-linear classifier**, like:
 - Decision Tree
 - Kernel SVM
 - Neural Network
- Train and Evaluate**

Example:

- Imagine classifying whether a person is fit based on:
- Calories consumed (x)
- Exercise done (y)
- Here, a curved boundary like a **circle** or **parabola** may better separate Fit vs Not Fit.

x (Calories)	y (Exercise)	Label
3000	30 min	Not Fit
2000	60 min	Fit
4000	20 min	Not Fit
2500	90 min	Fit

4. Multi-class Classification

Multi-class classification is used when there are **more than two classes**, and each example belongs to **only one class**.

Procedure:

- **Collect data** with multiple categories
- **Choose a classifier** that supports multi-class (e.g., Softmax Logistic Regression, Decision Tree)
- **Train and predict**

Example:

- Classifying **types of fruits** based on features:
- Model learns to predict whether a fruit is **Apple, Banana, or Orange**.

Color	Size	Shape	Label
Red	Small	Round	Apple
Yellow	Medium	Long	Banana
Orange	Medium	Round	Orange

Example:

- Predicting tags for a **YouTube video** based on title and description.
- A video can be both “Education” and “Programming” — hence, **multi-label**.

Title	Tags
“Python Tutorial”	[Programming, Education]
“Football Highlights”	[Sports, Entertainment]
“Cooking Pasta”	[Food, Lifestyle]

Types of Class	Description	Output Type	Example
Learning a Class	Learn to classify based on examples	Binary (+ or –)	Family car vs Not
Linear Classification	Linear decision boundary	Binary/Multiclass	Pass/Fail based on study hours
Non-Linear Classification	Curved/complex boundary	Binary/Multiclass	Fit/Not Fit based on exercise/calories
Multi-class Classification	Multiple exclusive classes	One class per sample	Apple, Banana, Orange
Multi-label Classification	Multiple non-exclusive labels	Many labels per sample	Tags: Education + Programming

Vapnik-Chervonenkis (VC) Dimension

1. Introduction to VC Dimension

- The **VC dimension** is a fundamental concept in **statistical learning theory**.
- It measures the **capacity or expressiveness** of a hypothesis class H .
- Introduced by **Vladimir Vapnik** and **Alexey Chervonenkis**, it helps us understand the **learning ability** of a model.

2. Shattering and Hypothesis Class

- Consider a dataset with N points.
- Each point can be labeled as either **positive or negative**, yielding 2^N possible labelings.
- If for **every possible labeling**, there exists a **hypothesis $h \in H$** that can **perfectly separate** the labeled data, then we say that **H shatters the set of N points**.
- In other words, H is powerful enough to **represent all possible classifications** of those N points.

3. Definition of VC Dimension

- The **VC dimension** of H , denoted by $VC(H)$, is the **largest number of points** that can be shattered by hypotheses in H .
- It gives a **quantitative measure of the capacity** of the hypothesis class.
- **Formally:**
- $VC(H)$ = maximum number of points that can be shattered by H

4. Example: Axis-Aligned Rectangles in 2D

- Let H be the class of **axis-aligned rectangles** in 2D.
- You can find **4 points** in the plane such that every possible labeling of them can be realized using some rectangle.
- Therefore, **$VC(H) = 4$** for axis-aligned rectangles in two dimensions.

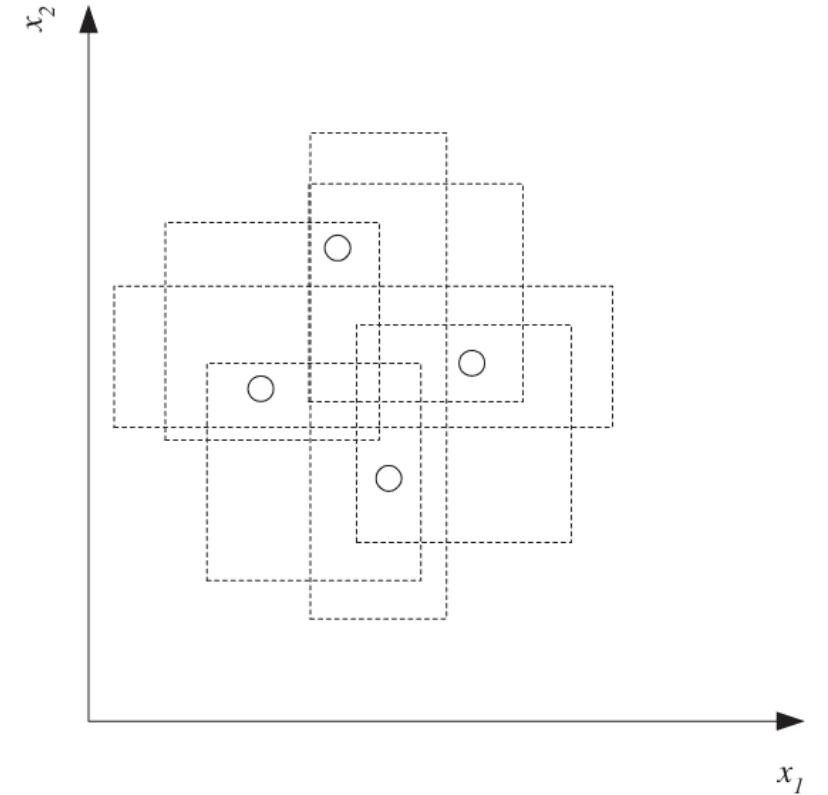


Fig: An axis-aligned rectangle can shatter four points.
Only rectangles covering two points are shown.

- (Figure shows a few rectangles covering only some points to represent different labelings.)
- Note: It's not required that any set of 4 points be shatterable—just one specific arrangement of 4 points is sufficient.
- However, 5 points cannot be shattered by axis-aligned rectangles in 2D. Thus, $VC(H) = 4$.

5. Practical Interpretation

- The VC dimension might seem **pessimistic**. For instance, if $VC(H) = 4$, it seems like the hypothesis can only handle 4-point datasets.
- However, this is a **worst-case bound**. It does **not** mean you can't learn from larger datasets.
- In practice:
 - Data often has **structure and regularity**.
 - **Not all labelings** are equally likely.
 - **Nearby points often have the same label** (smoothness assumption).

6. Usefulness of Small VC Dimension

- A **small VC dimension** implies a simpler hypothesis class.
- Simple models tend to have **better generalization** and are **less prone to overfitting**.
- In contrast, complex hypothesis classes (like lookup tables) may have **infinite VC dimension** and poor generalization.
- Therefore, **models with smaller VC dimensions** are often **preferred**, even though they can't shatter large point sets.

- VC dimension helps assess the **learning capacity** of a hypothesis class.
- It sets a bound on the **complexity** of patterns a model can learn.
- A **low VC dimension** model is not necessarily bad—it can still learn effectively in **real-world scenarios**.
- The VC dimension is **distribution-free**, making it a **conservative measure**.

Shattering	Ability of a hypothesis class to represent all labelings of a point set
VC Dimension	Maximum number of points that can be shattered
High VC Dimension	More complex model, may overfit
Low VC Dimension	Simpler model, better generalization
Axis-Aligned Rectangle (2D)	Has VC dimension of 4

Decision Trees

1. Introduction

- A **Decision Tree** is a **hierarchical data structure** that implements the **divide-and-conquer** strategy.
- It is a **nonparametric method**, meaning it does **not assume a fixed form** for the data distribution.
- ***Decision trees can be used for both:***
 - **Classification** (predicting categories)
 - **Regression** (predicting numeric values)

2. Parametric vs Nonparametric Estimation

- **Parametric Estimation:**

- A single global model is defined over the entire input space.
- Parameters of the model are learned from all the training data.
- The same model and parameters are used for all test inputs.

- **Nonparametric Estimation:**

- The input space is divided into **local regions**.
- A separate model is applied to each region based on **local training data**.
- Example: Distance-based models like **k-NN** use Euclidean distance, which requires **$O(N)$** computations for each prediction.

3. Decision Tree Structure

- A decision tree is composed of:
 - **Decision Nodes (Internal nodes)**: Apply a test or condition.
 - **Leaf Nodes (Terminal nodes)**: Contain the final output (class label or value).

3.1 Decision Node:

- Contains a **test function** $f_m(x)$, e.g., $x_1 < 5$
- Based on the outcome of the test, a specific **branch** is followed.
- Tests are applied sequentially from the **root** to a **leaf**.

3.2 Leaf Node:

- The final node reached after tests.
- Stores the **output**:
 - Class label (for classification)
 - Numeric value (for regression)

4. Learning a Decision Tree

- The tree is built recursively by:
 - Splitting the input space at each node using a **simple test**.
 - Refining the decision boundary with each subsequent level.
- Each test function $f_m(x)$ splits the **d-dimensional input space**.
- This continues until:
 - A **pure region** is found (all instances belong to one class), or
 - A stopping criterion is met (e.g., max depth, minimum samples).
- The **tree structure is learned from data**; it is **not fixed in advance**.

5. Characteristics of Decision Trees

Feature	Explanation
Nonparametric	No predefined model or parameters
Efficient	Fast prediction by eliminating cases at each node
Interpretable	Can be translated to IF-THEN rules
Localized Decisions	Breaks down complex decisions into smaller regions
Flexible	Tree size grows with data complexity

6. Univariate vs Multivariate Trees

- **Univariate Trees:**

- Each decision node uses only **one input variable**.
- Example: Test if $x_1 < 10$

- **Multivariate Trees:**

- Decision node uses **multiple input variables** in a combined test.
- Example: $3x_1 + 2x_2 < 5$
- Univariate trees are easier to interpret, while multivariate trees can be more compact and accurate.

7. Example and Interpretation

- A dataset is split using simple rules (e.g., axis-aligned splits).
- Each node splits the space orthogonally (perpendicular axes).
- If a region becomes **pure**, it is **not split further**.
- The tree can be **read from top to bottom** to trace the classification or regression path.

8. Advantages of Decision Trees

- **Fast classification and regression**
- **Interpretable structure** (can explain predictions)
- **Suitable for:**
 - **Medical applications** (clear reasoning required)
 - **Educational purposes** (easy to understand)
 - **Feature selection** (identifying important variables)

Concept	Description
Decision Tree	Hierarchical, nonparametric learning model
Decision Node	Applies test to input and routes the instance
Leaf Node	Contains output value or class
Univariate Tree	Each node uses one feature
Multivariate Tree	Nodes use combinations of features
IF-THEN Rules	Tree can be translated into readable logic
Efficiency	Requires fewer operations than instance-based models

Univariate Trees

1. What is a Univariate Tree?

- A **univariate decision tree** is a type of decision tree where:
 - Each **internal node** uses **only one input variable** (feature) to perform a test.
 - This simplifies the structure of the decision tree and makes it more interpretable.

2. Types of Input Variables and Splits

2.1 Discrete Inputs

- If the chosen feature x_j is categorical (e.g., $\text{color} \in \{\text{red}, \text{blue}, \text{green}\}$), then:
 - The decision node creates multiple branches — one for each category.
 - This is called an n -way split if the feature takes on n values.
- **Example:**
 - **Attribute:** color
 - **Possible values:** {red, blue, green}
 - The decision node will have three branches, each corresponding to one color.

2.2 Numeric Inputs

- If x_j is **numeric** (i.e., ordered), the test takes the form of a comparison

$$f_m(x) : x_j > w_{m0}$$

- w_{m0} is a threshold selected during learning.
- This creates a **binary split**:
 - **Left branch (L_m)**: $x_j > w_{m0}$
 - **Right branch (R_m)**: $x_j \leq w_{m0}$
- These binary splits **recursively divide** the input space at each level of the tree.
- Each new split is typically **orthogonal** to the previous one (i.e., along a different axis).

- **3. Visual Representation**

- As splits are made, the input space is divided into **hyperrectangles** (rectangular subregions in higher dimensions).
- Each **leaf node** of the tree corresponds to a **final region** in the input space with a specific class or value assigned.

- **4. Tree Induction (Tree Construction)**
- **Goal:**
- Given a labeled training dataset, construct a decision tree that classifies the data **accurately**.
- Many different trees can be constructed with **zero training error**, but we aim to find the **smallest tree** that still performs well.
- **Tree Size:**
- Measured by:
 - **Number of nodes**
 - **Complexity of decision functions** at each node
- **Challenge:**
- **Finding the smallest possible tree** is **NP-complete** (as proved by Quinlan, 1986).
- So we rely on **heuristic methods** and **greedy algorithms** to construct reasonably small trees in reasonable time.

- **5. Tree Building Process (Greedy Algorithm)**

- Start at the **root node** with the full training dataset.

- **At each node:**

- Choose the **best attribute** and, if numeric, a threshold to **split** the data.
- Perform the split (binary for numeric, n-way for discrete).
- Send the resulting subsets to **child nodes**.
- Repeat the process **recursively** on each subset.

- **Stopping condition:**

- When no further splitting is required (e.g., all samples have the same label, or another stopping rule is met), create a **leaf node** and assign a label/value.

Feature	Description
Univariate Tree	Each internal node uses only one feature for splitting
Discrete Attribute	Creates an n-way split; one branch per category
Numeric Attribute	Binary split using threshold $x_j > w_{m0}$
Tree Induction	Process of building the tree from labeled data
Tree Size	Number of nodes and complexity of decisions
Heuristic Search	Required due to NP-completeness of optimal tree search
Greedy Algorithm	Selects best split at each node recursively

7. Advantages of Univariate Trees

- Simple to interpret
- Easy to convert into readable IF-THEN rules
- Efficient when input variables are clearly separable

What Is Generalization Error?

- **Generalization error** is the difference between:
- **Empirical (training) error**: Error on the training dataset.
- **True error (expected error)**: Error on the entire distribution of data (including unseen data).

VC Dimension: A Measure of Model Complexity

- The **VC (Vapnik–Chervonenkis) Dimension** is a measure of the capacity or complexity of a hypothesis class H . It tells us **how expressive** a set of functions is.
- A hypothesis class **with higher VC dimension** can represent more complex patterns.
- But **too high** VC dimension $\rightarrow$ Overfitting.
- **Too low** VC dimension $\rightarrow$ Underfitting.
- ***VC Dimension = The largest number of points that can be shattered by the hypothesis class.***

Shattering

- A set of points is **shattered** by a hypothesis class if, for **every possible labeling (classification)** of those points, there exists a hypothesis in the class that classifies them correctly.

Example: VC Dimension of Linear Classifiers in 2D

- Consider 3 points in the plane:
 - No matter how you label them (8 combinations), there's always a line that can separate them correctly.
 - So, **3 points can be shattered** by linear classifiers in 2D.
- But 4 points:
 - For some arrangements (like XOR pattern), no line can separate all possible labelings.
 - So, **4 points cannot always be shattered**.
- **VC Dimension of linear classifiers in 2D = 3**

Generalization Error Bound Using VC Dimension

Let:

- $h \in \mathcal{H}$: hypothesis
- $L(h)$: true error (expected)
- $\hat{L}(h)$: training error (empirical)
- d : VC dimension of $\mathcal{H}$
- N : number of training examples
- δ : confidence level (e.g., 0.05 for 95%)

This means: **True error is at most the training error plus a complexity penalty term** that shrinks as N increases.

Then, with high probability $(1 - \delta)$, the generalization error is bounded as:

$$L(h) \leq \hat{L}(h) + \sqrt{\frac{d \cdot \log\left(\frac{2N}{d}\right) + 1}{N} + \log\left(\frac{4}{\delta}\right)}$$

- As sample size N increases $\rightarrow$ the bound becomes tighter (better generalization).
- As VC dimension d increases $\rightarrow$ the bound becomes looser (risk of overfitting).
- Lower VC dimension models generalize better on small datasets.

Numerical Example

Suppose:

- VC Dimension $d = 10$
- Training examples $N = 1000$
- Training error $\hat{L}(h) = 0.05$ (5%)
- Confidence level $\delta = 0.05$

Then, generalization error is approximately:

$$\begin{aligned} L(h) &\leq 0.05 + \sqrt{\frac{10(\log(200) + 1) + \log(80)}{1000}} \\ &\approx 0.05 + \sqrt{\frac{10(5.3 + 1) + 4.38}{1000}} = 0.05 + \sqrt{\frac{65.38 + 4.38}{1000}} = 0.05 + \sqrt{0.06976} \\ &\approx 0.05 + 0.26 = 0.31 \end{aligned}$$

So, with 95% confidence, the true error is at most 31%, though the training error is just 5%. This gap shrinks as you increase training data or reduce VC dimension.

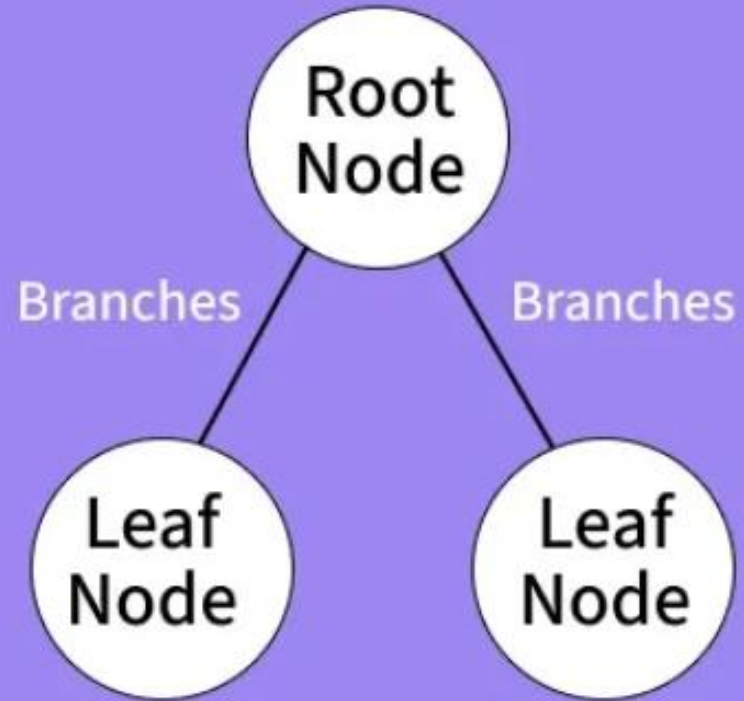
- VC Dimension helps us understand how likely our model is to generalize well.
- It acts as a regularization guide: don't use overly complex models for small data.
- Low training error does NOT guarantee low generalization error unless the model complexity is controlled.
- Always trade off between model complexity (VC dim) and amount of data (N) to ensure good generalization.

Decision Tree

- A Decision Tree helps us to make ***decisions by mapping out different choices and their possible outcomes.***
- It's used in machine learning for tasks like ***classification and prediction.***
- A Decision Tree helps us make decisions by showing different options and how they are related.
- It has a ***tree-like structure that starts with one main question called the root node*** which represents the entire dataset. From there, the tree branches out into different possibilities based on ***features in the data.***

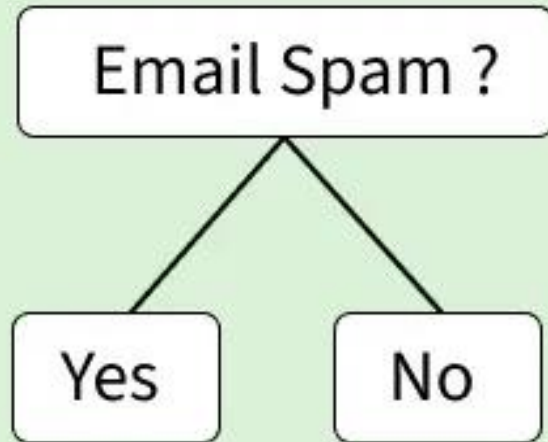
What is a Decision Tree?

- A Decision Tree maps out decisions and their outcomes.
- Starts with a root node and branches out into decisions.



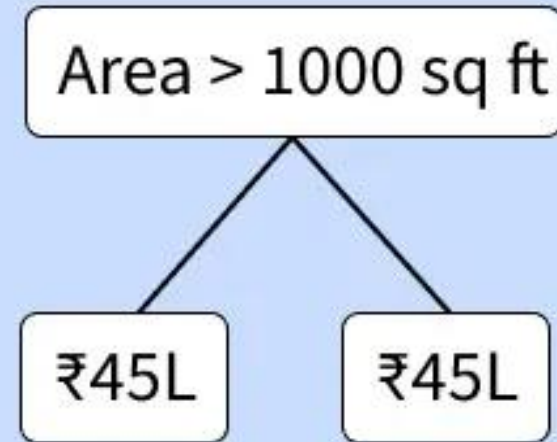
- **Root Node:** Starting point representing the whole dataset.
- **Branches:** Lines connecting nodes showing the flow from one decision to another.
- **Internal Nodes:** Points where decisions are made based on data features.
- **Leaf Nodes:** End points of the tree where the final decision or prediction is made.

Classification Tree



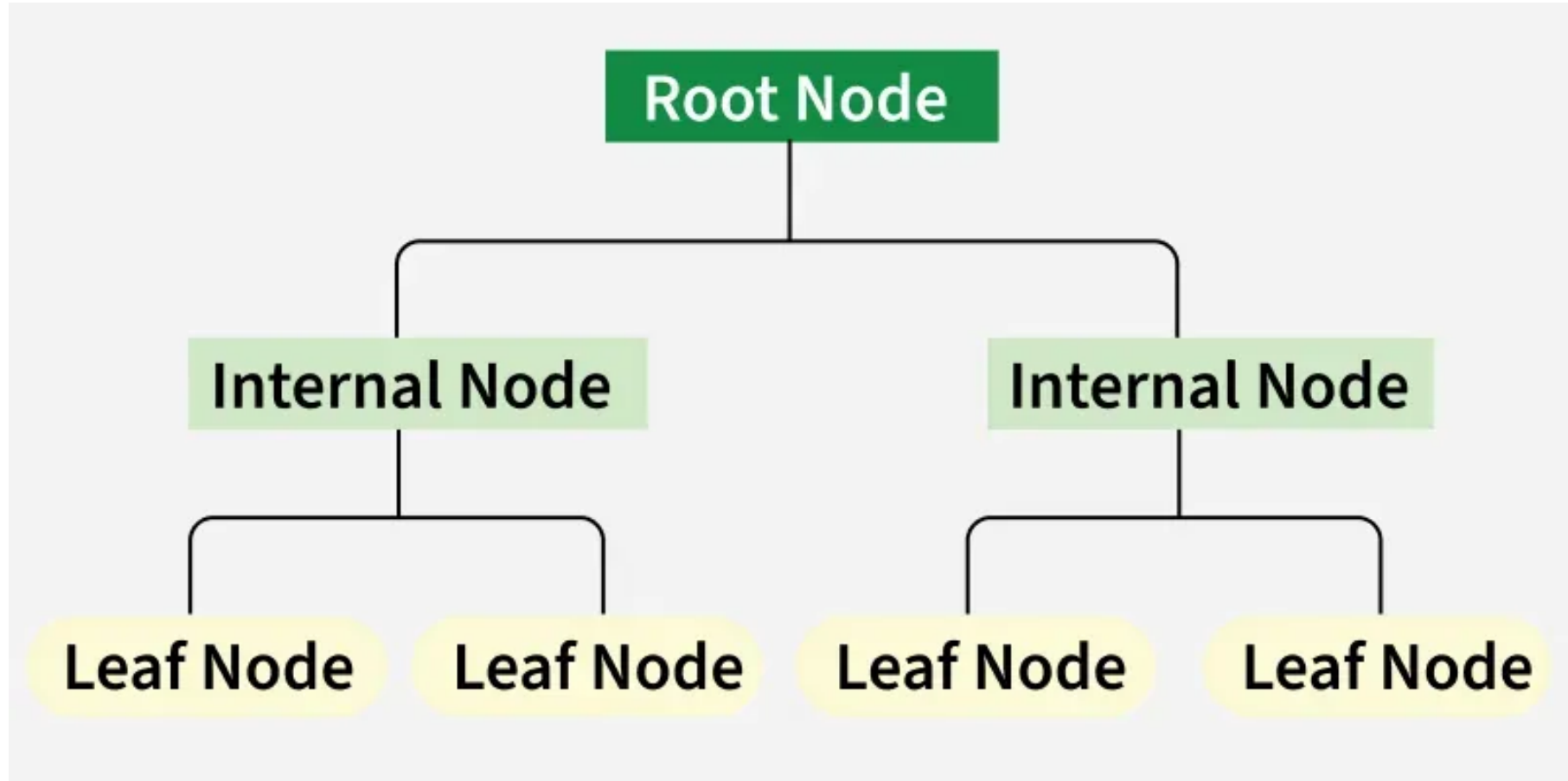
Yes/No decision icons.

Regression Tree



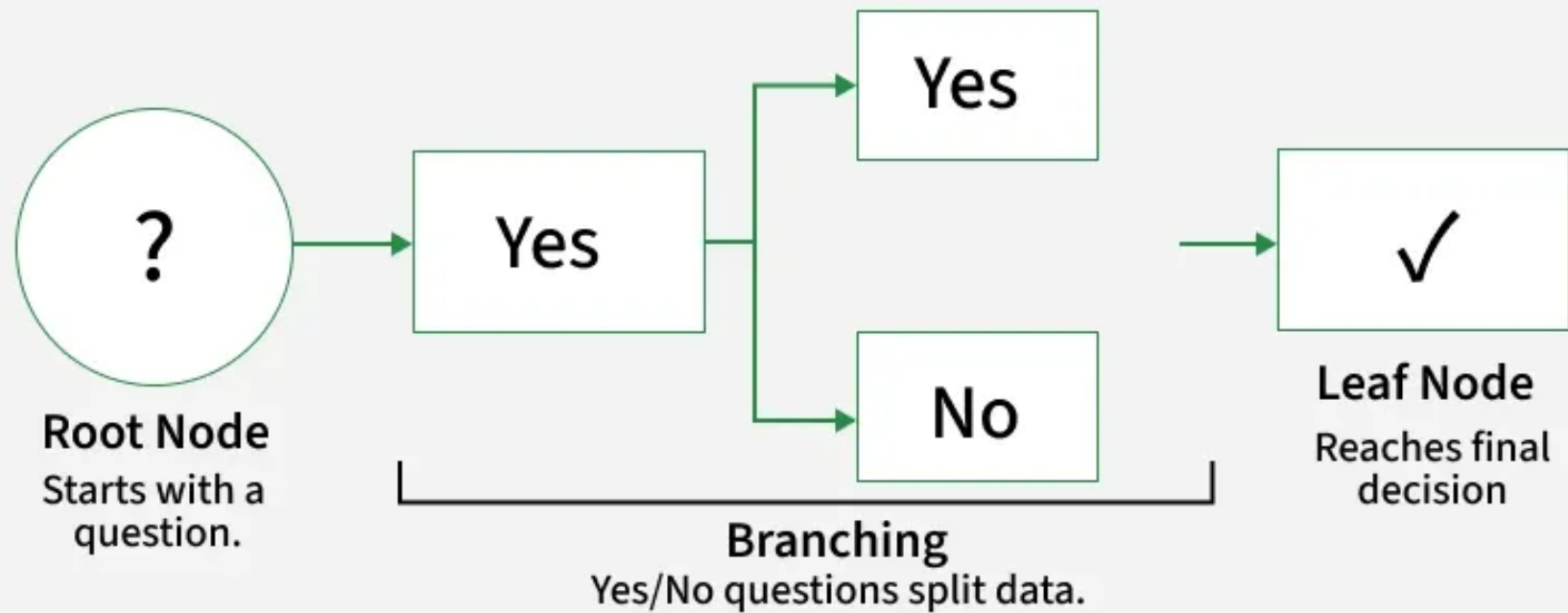
Numeric split at each node.

Types of Decision Trees



- A Decision Tree also helps with decision-making ***by showing possible outcomes clearly***. By looking at the "branches" we can quickly compare options and figure out the best choice.
- ***There are mainly two types of Decision Trees based on the target variable:***
- **Classification Trees:** Used for predicting ***categorical outcomes*** like spam or not spam. These trees split the data based on features to classify data into predefined categories.
- **Regression Trees:** ***Used for predicting continuous outcomes like predicting house prices***. Instead of assigning categories, it provides ***numerical predictions based on the input features***.

How Decision Trees Work?



Working of Decision Tree

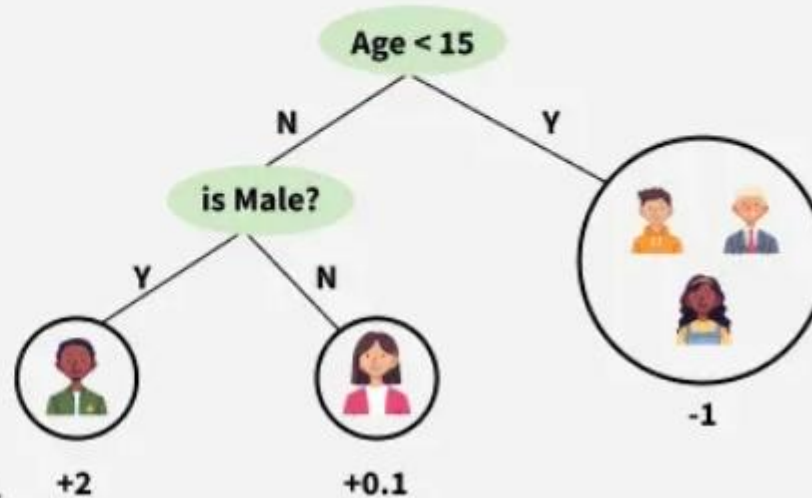
1. **Start with the Root Node:** It begins with a main question at the root node which is derived from the dataset's features.
2. **Ask Yes/No Questions:** From the root, the tree asks a series of yes/no questions to split the data into subsets based on specific attributes.
3. **Branching Based on Answers:** Each question leads to different branches:
 - If the answer is yes, the tree follows one path.
 - If the answer is no, the tree follows another path.
4. **Continue Splitting:** This branching continues through further decisions helps in reducing the data down step-by-step.
5. **Reach the Leaf Node:** The process ends when there are no more useful questions to ask leading to the leaf node where the final decision or prediction is made.

Working of Decision Tree

The model checks conditions like age and gender to split users into groups. Each group (leaf node) gets a prediction score based on user preferences for computer games.

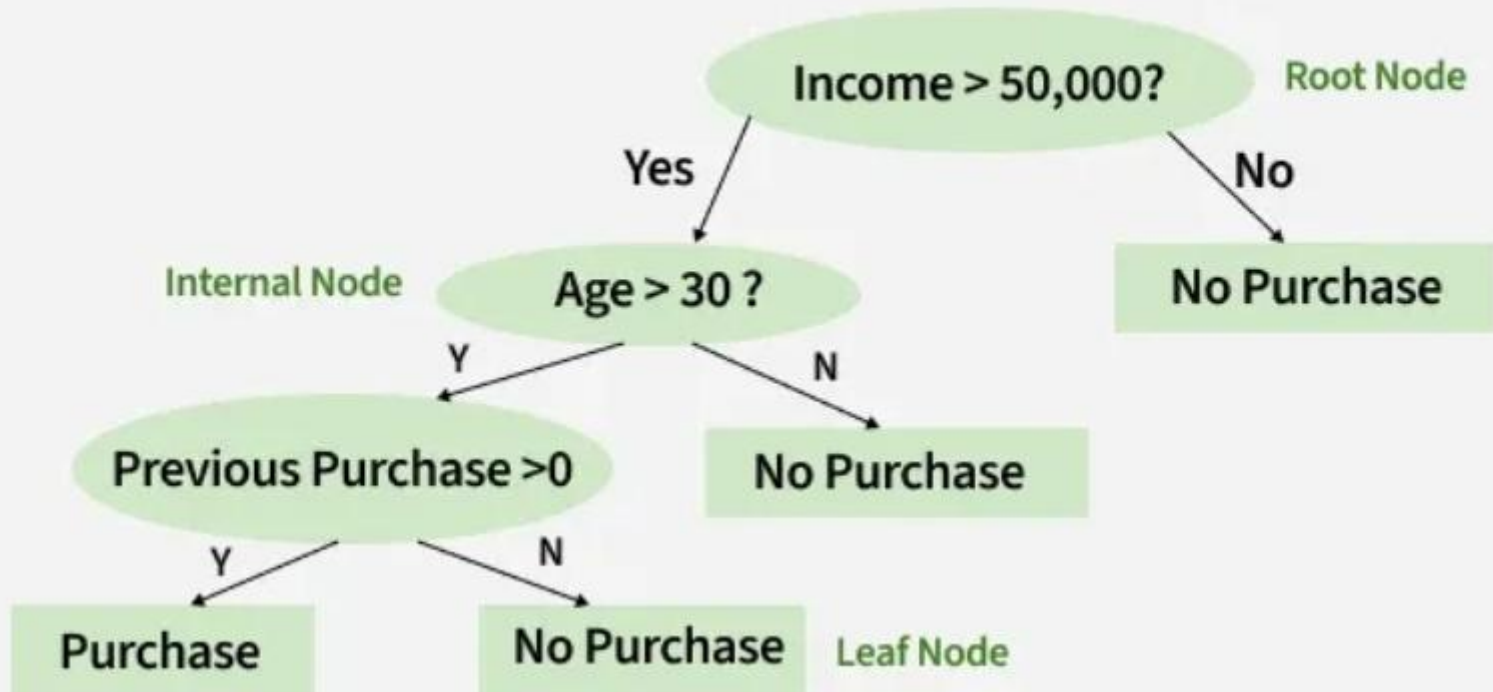
Input: Age, Gender, Occupation,...

Does the person likes computer games



Prediction score in each leaf →

Predicting whether a customer will buy a product



Splitting Criteria in Decision Trees

- In a Decision Tree, the process of splitting data at each node is important. The splitting criteria finds the best feature to split the data on. Common splitting criteria include [Gini Impurity and Entropy](#).
- **Gini Impurity:** This criterion measures how "impure" a node is. The lower the Gini Impurity the better the feature splits the data into distinct categories.
- **Entropy:** This measures the amount of uncertainty or disorder in the data. The tree tries to reduce the entropy by splitting the data on features that provide the most information about the target variable.
- *These criteria help decide which features are useful for making the best split at each decision point in the tree.*

Gini Index

$$I_G = 1 - \sum_{j=1}^c p_j^2$$

p_j : proportion of the samples that belongs to class c for a particular node

Entropy

$$I_H = - \sum_{j=1}^c p_j \log_2(p_j)$$

p_j : proportion of the samples that belongs to class c for a particular node.

*This is the the definition of entropy for all non-empty classes ($p \neq 0$) The entropy is 0 if all samples at a node belong to the same class.

Entropy

- Entropy helps us to build an appropriate decision tree for selecting the best splitter.
- Entropy can be defined as a measure of the purity of the sub-split. Entropy always lies between 0 to 1. The entropy of any split can be calculated by this formula.

$$H(s) = -P_{(+)} \log_2 P_{(+)} - P_{(-)} \log_2 P_{(-)}$$

Here $\frac{P_{(+)}}{P_{(+) + P_{(-)}}} =$
% of positive class, and 1% of negative class.

The algorithm calculates the entropy of each feature after every split and as the splitting continues on, it selects the best feature and starts splitting according to it.

Gini Impurity

- The internal working of Gini impurity is also somewhat similar to the working of entropy in the Decision Tree.
- In the Decision Tree algorithm, both are used for building the tree by splitting as per the appropriate features but there is quite a difference in the computation of both methods.
- Gini Impurity of features after splitting can be calculated by using this formula.

$$GI = 1 - \sum_{i=1}^n (p_i)^2 \quad GI = 1 - \left[(P_{(+)})^2 + (P_{(-)})^2 \right]$$

Difference between Gini Index and Entropy

It is the probability of misclassifying a randomly chosen element in a set.	While entropy measures the amount of uncertainty or randomness in a set.
The range of the Gini index is [0, 0.5] , where 0 indicates perfect purity and 0.5 indicates maximum impurity.	The range of entropy is [0, log₂(C)] , where c is the number of classes. The range becomes [0, 1] for binary classification.
Gini index is a linear measure.	Entropy is a logarithmic measure.
It can be interpreted as the expected error rate in a classifier.	It can be interpreted as the average amount of information needed to specify the class of an instance.
It is sensitive to the distribution of classes in a set.	It is sensitive to the number of classes.
The computational complexity of the Gini index is O(c).	Computational complexity of entropy is O(c * log(c)).
It is less robust than entropy.	It is more robust than Gini index.
It is sensitive.	It is comparatively less sensitive.
Formula for the Gini index is $Gini = 1 - \sum_{x=1}^C p_x^2$, where p_x is the proportion of the instances of class x in a set.	Formula for entropy is $Entropy = - \sum_{x=1}^C p_x \log_2(p_x)$, where p_x is the proportion of the instances of class x in a set.
It has a bias toward selecting splits that result in a more balanced distribution of classes.	It has a bias toward selecting splits that result in a higher reduction of uncertainty.
Gini index is typically used in CART (Classification and Regression Trees) algorithms	Entropy is typically used in ID3 and C4.5 algorithms

01

Loan Approval

02

Medical Diagnosis

03

Predicting
Exam Results

Applications of Decision Trees

Customer Churn
Prediction

06

Fraud Detection

05

Marketing
Campaigns

04

Example: Now let us draw a Decision Tree for the following data using Information gain. Training set: 3 features and 2 classes

X	Y	Z	C
1	1	1	I
1	1	0	I
0	0	1	II
1	0	0	II

- Features:** X, Y, Z
- Target (Class):** C (values: I or II)

- Step 1: Calculate EntWe have 4 samples:
- Class I: 2 samples
- Class II: 2 samples
- **Entropy formula:**

Entropy formula:

$$Entropy(S) = -p_1 \log_2(p_1) - p_2 \log_2(p_2)$$

Where:

- $p_1 = 2/4 = 0.5$ (for class I)
- $p_2 = 2/4 = 0.5$ (for class II)

$$Entropy(S) = -0.5 \log_2(0.5) - 0.5 \log_2(0.5) = 1$$

- Step 2: Calculate Information Gain for each feature
- **Split on Feature X**
- Possible values: $X = 0$ or 1

X	Y	Z	C
1	1	1	I
1	1	0	I
1	0	0	II
0	0	1	II

Subset 1 ($X = 1$) $\rightarrow$ 3 samples:

- I: 2, II: 1

$$Entropy = -\frac{2}{3} \log_2 \left(\frac{2}{3} \right) - \frac{1}{3} \log_2 \left(\frac{1}{3} \right) \approx 0.918$$

Subset 2 ($X = 0$) $\rightarrow$ 1 sample:

- II: 1 (pure class)

$$Entropy = 0$$

Weighted Entropy:

$$E_X = \frac{3}{4}(0.918) + \frac{1}{4}(0) = 0.688$$

Information Gain for X:

$$IG(X) = 1 - 0.688 = 0.312$$

- Split on Feature Y

Y

1

1

0

0

C

I

I

II

II

Subset 1 (Y = 1) → 2 samples:

- I: 2 (pure) → Entropy = 0

Subset 2 (Y = 0) → 2 samples:

- II: 2 (pure) → Entropy = 0

Weighted Entropy:

$$E_Y = \frac{2}{4}(0) + \frac{2}{4}(0) = 0$$

Information Gain for Y:

$$IG(Y) = 1 - 0 = 1$$

Maximum Gain!

- Split on feature Z

Z	C
1	I
0	I
1	II
0	II

Subset 1 ($Z = 1$) $\rightarrow$ 2 samples: I, II $\rightarrow$ Entropy = 1

Subset 2 ($Z = 0$) $\rightarrow$ 2 samples: I, II $\rightarrow$ Entropy = 1

Weighted Entropy:

$$E_Z = \frac{2}{4}(1) + \frac{2}{4}(1) = 1$$

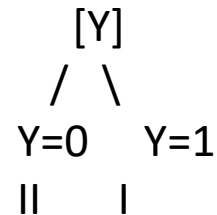
Information Gain for Z:

$$IG(Z) = 1 - 1 = 0$$

- Step 3: Choose the Feature with Highest Information Gain
- $IG(X) = 0.312$
- $IG(Y) = \mathbf{1.0}$
- $IG(Z) = 0$
- So, **feature Y** is chosen for the **root node**.

Step 4: Final Decision Tree

- **Split on Feature Y:**
- If $Y = 1 \rightarrow$ Class I (pure subset)
- If $Y = 0 \rightarrow$ Class II (pure subset)
- No further splitting is required because all child nodes are **pure**.



Draw the decision tree for the following dataset

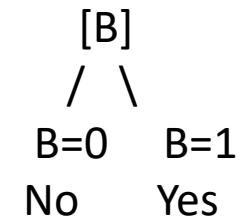
A	B	C	Class
1	1	0	Yes
1	0	0	No
0	1	1	Yes
0	0	1	No
1	1	1	Yes

Feature	Info Gain
A	0.020
B	0.971
C	0.0202

Step 4: Build the Tree

Split on Feature B:

- If **B = 1** → Samples: [Yes, Yes, Yes] → **All Yes** → **No more split**
- If **B = 0** → Samples: [No, No] → **All No** → **No more split**



ID3

- The ID3 (Iterative Dichotomiser 3) algorithm is a classic algorithm used in machine learning ***to construct decision trees for classification tasks.***
- It operates on the principle of information gain to determine the ***best attribute for splitting the data at each node of the tree.***

ID3 Metrics

1. Information Gain and Attribute Selection

- The ID3 algorithm uses a measure of impurity, such as entropy or Gini impurity, to calculate the **information gain** of each attribute.
- **Entropy** is a measure of disorder in a dataset.
- A dataset with high entropy is a dataset where the data points are evenly distributed across the different categories.
- A dataset with low entropy is a dataset where the data points are concentrated in one or a few categories.

What are the steps in ID3 algorithm?

1. **Determine entropy** for the overall the dataset using class distribution.
2. For each feature.
 - Calculate **Entropy for Categorical Values**.
 - Assess **information gain** for each unique categorical value of the feature.
3. Choose the feature that generates **highest information gain**.
4. Iteratively apply all above steps to build the decision tree structure.

Advantages of ID3

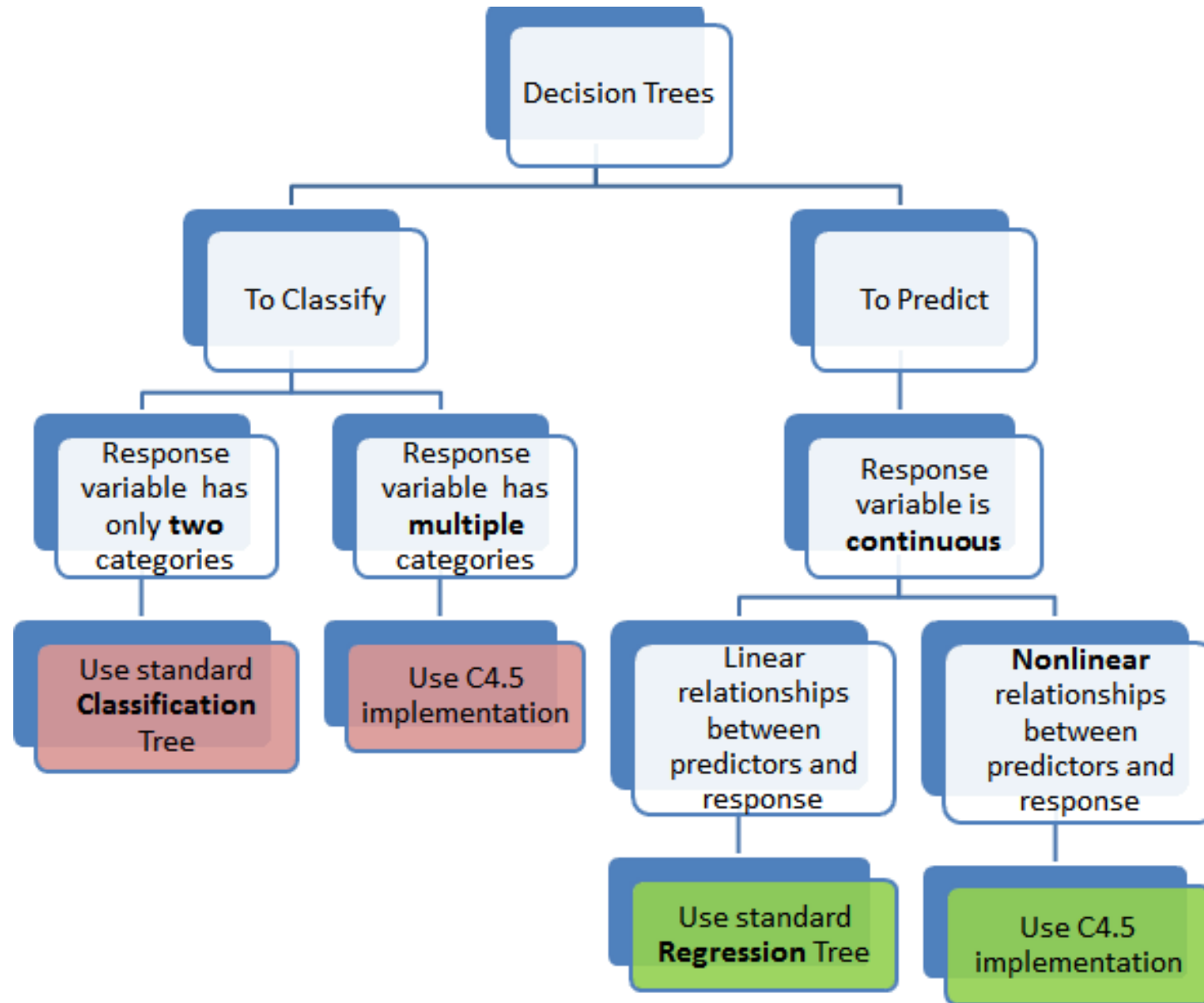
- Simple and easy to understand.
- Requires little training data.
- Can work well with data with discrete and continuous attributes.

Disadvantages of ID3

- Can lead to overfitting.
- May not be effective with data with many attributes.

Applications of ID3

- **Fraud detection:** ID3 can be used to develop models that can detect fraudulent transactions or activities.
- **Medical diagnosis:** ID3 can be used to develop models that can diagnose diseases or medical conditions.
- **Customer segmentation:** ID3 can be used to segment customers into different groups based on their demographics, purchase history, or other factors.
- **Risk assessment:** ID3 can be used to assess risk in a variety of different areas, such as insurance, finance, and healthcare.
- **Recommendation systems:** ID3 can be used to develop recommendation systems that can recommend products, services, or content to users based on their past behavior or preferences.
 - Amazon uses ID3 to recommend products to its customers.
 - Netflix uses ID3 to recommend movies and TV shows to its users.
 - Spotify uses ID3 to recommend songs and playlists to its users.
 - LendingClub uses ID3 to assess the risk of approving loans to borrowers.
 - Healthcare organizations use ID3 to diagnose diseases, predict patient outcomes, and develop personalized treatment plans.



Classification and Regression Trees (CART)

Concept	Classification Tree	Regression Tree
1. Output Type	Predicts categorical/class labels (e.g., Yes/No, A/B/C)	Predicts continuous numeric values (e.g., price, temperature)
2. Splitting Criteria	Uses Gini impurity, entropy, or information gain	Uses Mean Squared Error (MSE), Mean Absolute Error (MAE)
3. Leaf Node Output	Contains a class label or class probabilities	Contains a numerical value (e.g., average of the target values)
4. Use Cases	Used in classification problems (e.g., spam detection)	Used in regression problems (e.g., house price prediction)
5. Evaluation Metrics	Accuracy, Precision, Recall, F1-score, ROC-AUC	RMSE (Root Mean Square Error), MAE (Mean Absolute Error), R^2

1. Accuracy

- The ratio of correctly predicted instances (both positive and negative) to the total instances.
- If a model correctly predicts 90 out of 100 samples, accuracy = 90%.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

TP (True Positive)

- The model correctly predicts a **positive** class.
- Example: Model predicts "disease" and the person actually has the disease.

TN (True Negative)

- The model correctly predicts a **negative** class.
- Example: Model predicts "no disease" and the person is healthy.

P (False Positive) – Type I Error

- The model **incorrectly predicts positive** when it is actually negative.
- Example: Model predicts "disease" but the person is healthy.

FN (False Negative) – Type II Error

- The model **incorrectly predicts negative** when it is actually positive.
- Example: Model predicts "no disease" but the person has the disease.

2. Precision

- The ratio of correctly predicted positive observations to the total predicted positive observations.

Example:

If your model predicts 10 positives, out of which only 7 are actually positive, precision = $7/10 = 0.7$ (70%).

$$\text{Precision} = \frac{TP}{TP + FP}$$

3. Recall (Sensitivity / True Positive Rate)

- The ratio of correctly predicted positive observations to **all actual positives**.

Example:

- If there are 12 actual positives and your model detects 9 of them correctly, recall = ?

$$\text{Recall} = \frac{TP}{TP + FN}$$

4. F1-Score

- The harmonic mean of **precision** and **recall**. Balances the two when there is class imbalance.

Example:

If precision = 0.8 and recall = 0.6,

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{F1} = 2 \times \frac{0.8 \times 0.6}{0.8 + 0.6} = 0.6857$$

These four terms form the **confusion matrix**, and are used to calculate all major classification metrics like:

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

$$\text{Precision} = TP / (TP + FP)$$

$$\text{Recall} = TP / (TP + FN)$$

$$\text{F1 Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

- A **confusion matrix** is a table used to describe the performance of a **classification model** on a set of test data where the true values are known. It shows the number of **correct and incorrect predictions** made by the model, broken down by class.

5. ROC-AUC (Receiver Operating Characteristic - Area Under Curve)

- AUC measures the **entire two-dimensional area underneath the ROC curve**, which plots **True Positive Rate vs False Positive Rate**.
- **Interpretation:**
 - AUC = 1: Perfect classifier
 - AUC = 0.5: No discrimination (like random guessing)
- **Example:**

If your model has an AUC of 0.92, it means there's a 92% chance it will correctly distinguish between a positive and a negative class.

1. RMSE – Root Mean Square Error

- RMSE measures the **square root of the average of the squared differences** between predicted and actual values.

It gives **more weight to large errors**.

- **Example:**

Actual values: [3, 5, 2.5]

Predicted values: [2.5, 5, 4]

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

$$\text{RMSE} = \sqrt{((3 - 2.5)^2 + (5 - 5)^2 + (2.5 - 4)^2)/3} = \sqrt{(0.25 + 0 + 2.25)/3} = \sqrt{0.833} \approx 0.91$$

R² – Coefficient of Determination

- R² measures the **proportion of variance** in the dependent variable that is **predictable** from the independent variable(s).
- R² = 1 → perfect prediction
- R² = 0 → no predictive power
- R² < 0 → worse than a horizontal line

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$$

Where:

- $SS_{\text{res}} = \sum (y_i - \hat{y}_i)^2$
- $SS_{\text{tot}} = \sum (y_i - \bar{y})^2$

Example:

If R² = 0.92, it means **92% of the variance** in the actual data is explained by the model.

Regression

- **Regression** is a type of **supervised machine learning** technique used to **predict a continuous output value** based on one or more input (independent) variables.
- **Regression Equation:**

$$Y=f(X)+\varepsilon$$

- Where:
- Y: Dependent variable (target/output)
- X: Independent variable(s) (input)
- $f(X)$: Function that models the relationship (e.g., linear)
- ε : Error term (difference between prediction and actual)

Predict a student's **exam score (Y)** based on the **number of hours studied (X)**.

Hours Studied (X)	Marks Scored (Y)
1	52
2	56
3	61
4	66
5	71

Apply Regression Model:

- $Y = mX + c + \epsilon$
- Where:
- X: Hours studied
- Y: Marks scored
- m Slope of the line (rate of increase in marks per hour)
- c: Intercept (marks when $X = 0$)

In regression analysis, the **intercept** is the value of the **dependent variable (Y)** when **all independent variables (X) are zero**.

- ϵ : Error term (difference between actual and predicted)

$$Y=5X+47$$

Prediction Example:

If a student studies for **6 hours**, predict the marks:

$$Y=5(6)+47=30+47=77$$

To calculate the m

- Step 1: Calculate Mean of X and Y

$$\bar{X} = \frac{1 + 2 + 3 + 4 + 5}{5} = \frac{15}{5} = 3$$

$$\bar{Y} = \frac{52 + 56 + 61 + 66 + 71}{5} = \frac{306}{5} = 61.2$$

- Step 2: Calculate the Slope (m)

$$m = \frac{\sum (X - \bar{X})(Y - \bar{Y})}{\sum (X - \bar{X})^2}$$

X	Y	X - $\bar{X}$	Y - $\bar{Y}$	(X - $\bar{X}$)(Y - $\bar{Y}$)	(X - $\bar{X}$) ²
1	52	-2	-9.2	18.4	4
2	56	-1	-5.2	5.2	1
3	61	0	-0.2	0	0
4	66	1	4.8	4.8	1
5	71	2	9.8	19.6	4

$$m = \frac{18.4 + 5.2 + 0 + 4.8 + 19.6}{4 + 1 + 0 + 1 + 4} = \frac{48}{10} = 4.8$$

- Step 3: Calculate Intercept (c)

$$c = \bar{Y} - m \cdot \bar{X}$$

$$c = 61.2 - (4.8)(3) = 61.2 - 14.4 = 46.8$$

- Final Regression Line:
- $Y = 4.8X + 46.8$

1. Linear Regression

- Linear Regression is used to model the relationship between a **single independent variable (X)** and a **dependent variable (Y)** by fitting a straight line ($Y = mX + c$) to the data.

2. Multiple Linear Regression

- Used when there are **two or more independent variables** predicting a dependent variable.

$$Y = b_0 + b_1X_1 + b_2X_2 + \dots + b_nX_n$$

Predict the price of a house based on **size (sqft)** and **number of bedrooms**.

Size (X_1)	Bedrooms (X_2)	Price (Y)
1000	2	300,000
1500	3	400,000
2000	3	500,000
2500	4	600,000

- To calculate the predicted price Y using the multiple linear regression equation:

$$Y = b_0 + b_1(1800) + b_2(3)$$

- We need the actual **coefficients** b_0 , b_1 , and b_2 .

Coefficient	Value
b_0	20,000 (intercept)
b_1	200 (per sqft)
b_2	15,000 (per bedroom)

$$Y = 20,000 + 200 \times 1800 + 15,000 \times 3$$

$$Y = 20,000 + 360,000 + 45,000$$

$$Y = 425,000$$

$$Y = ₹425,000$$

Logistic Regression

- Used when the **dependent variable is categorical**, typically binary (0 or 1).

Instead of predicting the value directly, it predicts the **probability** that Y belongs to a certain class.

$$P(Y = 1) = \frac{1}{1 + e^{-(b_0 + b_1 X)}}$$

Predict whether a student passes (1) or fails (0) based on hours studied.

Hours Studied (X)

1

2

3

4

5

Pass (Y)

0

0

1

1

1

1. Assume the **logistic function**:

$$\text{Logit}(p) = \log \left(\frac{p}{1-p} \right) = b_0 + b_1 X$$

2. Fit the model using **maximum likelihood estimation (MLE)**.

3. Get the equation:

$$P(\text{pass}) = \frac{1}{1 + e^{-(b_0 + b_1 X)}}$$

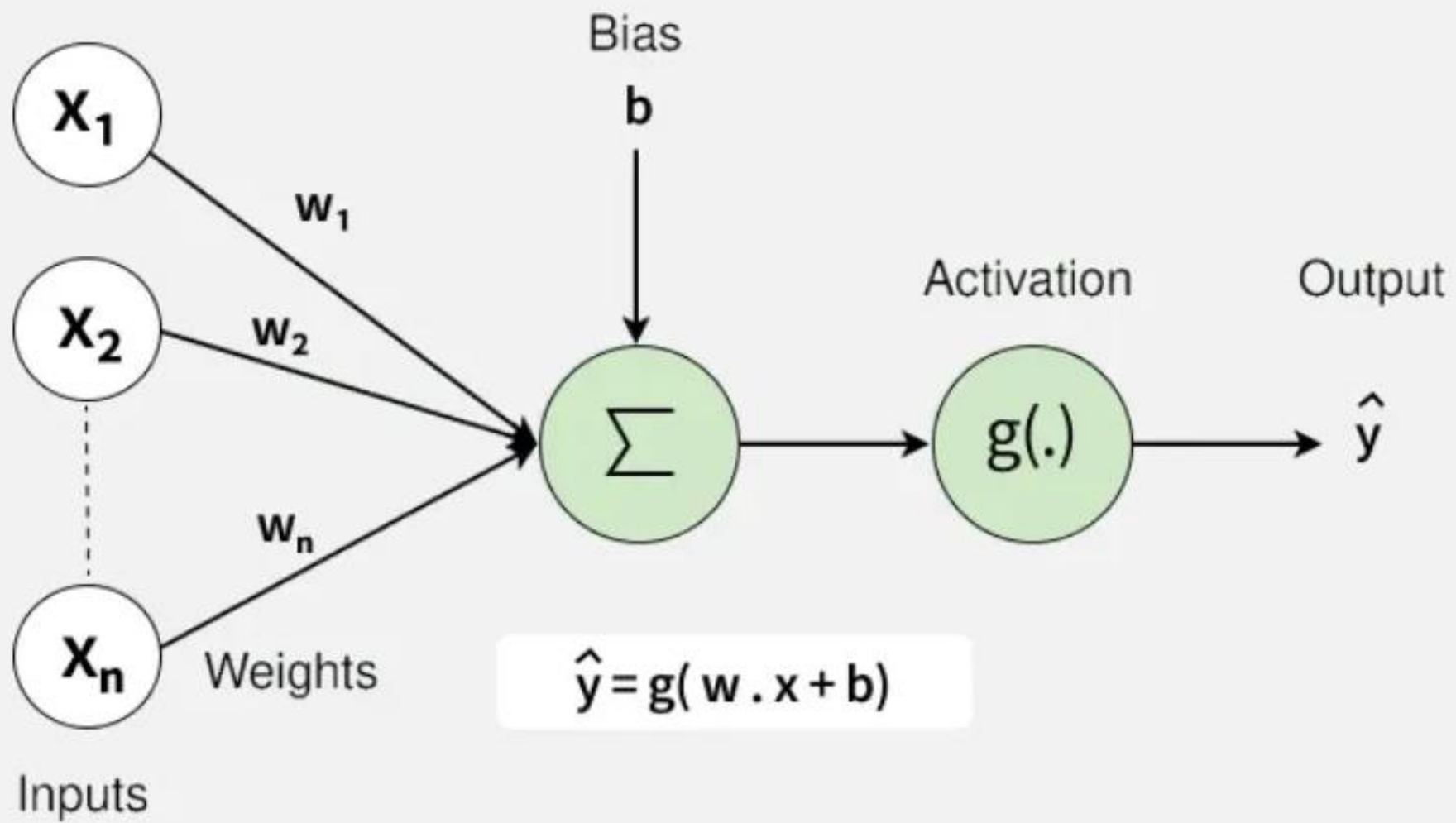
4. **Predict:**

- For a student who studied 3.5 hours, calculate the probability.
- If $P > 0.5$, classify as "Pass"; else "Fail".

Type	Dependent Variable	Independent Variables	Output
Linear Regression	Continuous	1	Continuous value
Multiple Linear Reg.	Continuous	2 or more	Continuous value
Logistic Regression	Categorical (0/1)	1 or more	Probability (0–1)

What is a Neural Network?

- Neural networks are **machine learning models** that mimic the complex functions of the human brain.
- These models consist of *interconnected nodes or neurons that process data, learn patterns* and enable tasks such as *pattern recognition and decision-making*.



Symbol	Meaning
$x_1, x_2, \dots, x_n$	Input features
$w_1, w_2, \dots, w_n$	Weights for each input
b	Bias
$\sum$	Weighted sum: $\mathbf{w} \cdot \mathbf{x} + b$
$g(z)$	Activation function
$\hat{y}$	Output prediction

- This diagram represents a **single neuron (perceptron)** in a **neural network**, showing how input data is processed to produce an output prediction.

$$\hat{y} = g(\mathbf{w} \cdot \mathbf{x} + b)$$

- This is the **forward propagation** formula for a single neuron.

1. Inputs ($x_1, x_2, \dots, x_n$)

- Represent the **features** or data given to the model.
- Example: In image recognition, each pixel value is an input.
- In a student's pass/fail prediction, x_1 might be "hours studied", x_2 could be "attendance", etc.

2. Weights ($w_1, w_2, \dots, w_n$)

- Each input is multiplied by a corresponding **weight**.
- Weights determine **how important** each input is.
- During training, the neural network **learns optimal weights** to make accurate predictions.

3. Weighted Sum (Σ)

- The neuron **calculates the dot product** of weights and inputs:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n \Rightarrow \mathbf{w} \cdot \mathbf{x}$$

- This result is called the **linear combination**.

4. Bias (b)

- A **bias term** is added to the weighted sum to **shift** the activation function.
- Helps the model learn even when all input values are 0.
- Makes the model **more flexible**.

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

Bias is a trainable constant added to the weighted sum of inputs **before** applying the activation function.

5. Activation Function (g(.))

- Applies a **non-linear transformation** to the input.
- Most commonly used activation in classification tasks: **Sigmoid function**:

$$g(z) = \frac{1}{1 + e^{-z}}$$

Rectified Linear Unit

Hyperbolic Tangent

- Other activations: ReLU, Tanh, etc.
- Without this, the network would behave like a linear model.

6. Output ($\hat{y}$ or y-hat)

- This is the **final output** from the neuron.
- For binary classification (like pass/fail), it's a probability:

$$\hat{y} = g(\mathbf{w} \cdot \mathbf{x} + b)$$

- Value ranges between **0 and 1** (if using sigmoid), representing the **probability of class 1**.

Why is Bias Important?

Variation	Bias allows the neuron's activation function to shift left or right , making it more flexible.
Useful Learning	Even if all input values are zero , the bias helps produce a meaningful output.
Precision	Without bias, the neuron can only respond in a limited way. Adding bias makes the neuron more powerful and capable .

Multi-Layer Perceptron

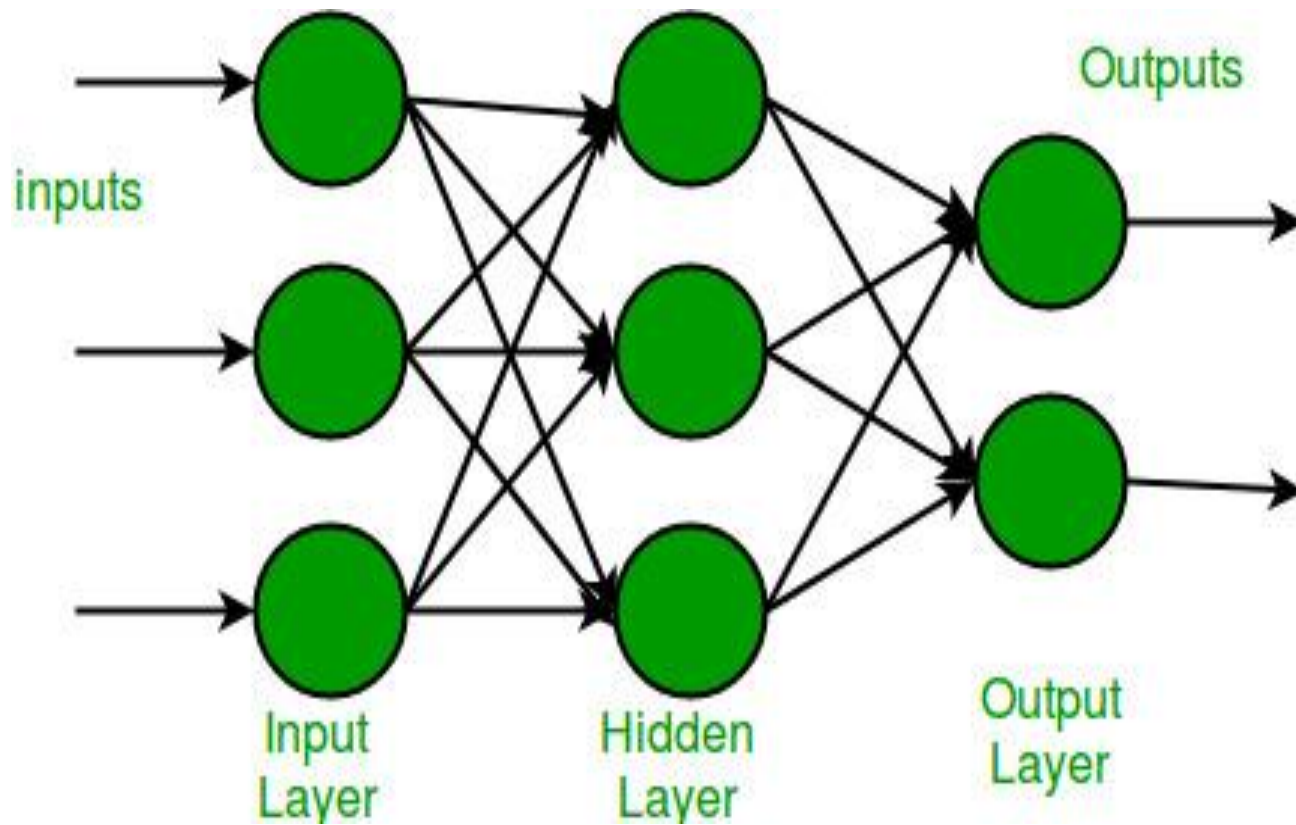
- **Multi-Layer Perceptron (MLP)** consists of fully connected dense layers that transform input data from one dimension to another.
- It is called **multi-layer** because it contains an input layer, one or more hidden layers and an output layer.
- ***The purpose of an MLP is to model complex relationships between inputs and outputs.***

Components of Multi-Layer Perceptron (MLP)

- **Input Layer:** Each neuron or node in this layer corresponds to an input feature. For instance, if you have three input features the input layer will have three neurons.
- **Hidden Layers:** MLP can have any number of hidden layers with each layer containing any number of nodes. These layers process the information received from the input layer.
- **Output Layer:** The output layer generates the ***final prediction or result***. If there are multiple outputs, the output layer will have a corresponding number of neurons.

Components of Multi-Layer Perceptron (MLP)

- **Input Layer:** Each neuron or node in this layer corresponds to an input feature. For instance, if you have three input features the input layer will have three neurons.
- **Hidden Layers:** MLP can have any number of hidden layers with each layer containing any number of nodes. These layers process the information received from the input layer.
- **Output Layer:** The output layer generates the final prediction or result. If there are multiple outputs, the output layer will have a corresponding number of neurons.



- Every connection in the diagram is a representation of the fully connected nature of an MLP.
- ***This means that every node in one layer connects to every node in the next layer.***
- As the data moves through the network each layer transforms it until the final output is generated in the output layer.

Working of Multi-Layer Perceptron

1. Forward Propagation

In **forward propagation** the data flows from the input layer to the output layer, passing through any hidden layers. Each neuron in the hidden layers processes the input as follows:

1. Weighted Sum: The neuron computes the weighted sum of the inputs:

$$z = \sum_i w_i x_i + b$$

Where:

- x_i is the input feature.
- w_i is the corresponding weight.
- b is the bias term.

2. Activation Function: The weighted sum z is passed through an activation function to introduce non-linearity. Common activation functions include:

- Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$
- ReLU (Rectified Linear Unit): $f(z) = \max(0, z)$
- Tanh (Hyperbolic Tangent): $\tanh(z) = \frac{2}{1+e^{-2z}} - 1$

2. Loss Function

Once the network generates an output the next step is to calculate the loss using a loss function. In supervised learning this compares the predicted output to the actual label.

For a classification problem the commonly used binary cross-entropy loss function is:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Where:

- y_i is the actual label.
- $\hat{y}_i$ is the predicted label.
- N is the number of samples.

For regression problems the mean squared error (MSE) is often used:

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

3. Backpropagation

The goal of training an MLP is to minimize the loss function by adjusting the network's weights and biases. This is achieved through backpropagation:

1. **Gradient Calculation:** The gradients of the loss function with respect to each weight and bias are calculated using the chain rule of calculus.
2. **Error Propagation:** The error is propagated back through the network, layer by layer.
3. **Gradient Descent**: The network updates the weights and biases by moving in the opposite direction of the gradient to reduce the loss: $w = w - \eta \cdot \frac{\partial L}{\partial w}$

Where:

- w is the weight.
- η is the learning rate.
- $\frac{\partial L}{\partial w}$ is the gradient of the loss function with respect to the weight.

4. Optimization

MLPs rely on optimization algorithms to iteratively refine the weights and biases during training. Popular optimization methods include:

- **Stochastic Gradient Descent (SGD)**: Updates the weights based on a single sample or a small batch of data: $w = w - \eta \cdot \frac{\partial L}{\partial w}$
- **Adam Optimizer**: An extension of SGD that incorporates momentum and adaptive learning rates for more efficient training:
 - $m_t = \beta_1 m_{t-1} + (1 - \beta_1) \cdot g_t$
 - $v_t = \beta_2 v_{t-1} + (1 - \beta_2) \cdot g_t^2$
- Here g_t represents the gradient at time t and β_1, β_2 are decay rates.

Support Vector Machine (SVM)

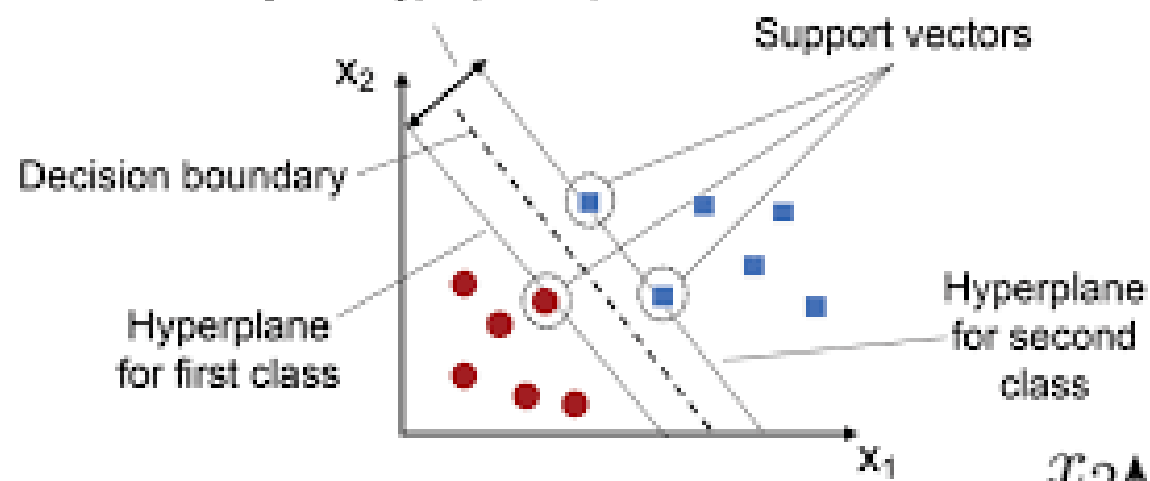
- Support Vector Machine (SVM) is a supervised machine learning algorithm used for *classification and regression* tasks.
- It tries to **find the best boundary known as hyperplane that separates different classes in the data.**
- It is useful when you want to do binary classification like spam vs. not spam or cat vs. dog.
- The main goal of SVM is *to maximize the margin between the two classes.*
- The larger *the margin the better the model performs on new and unseen data.*

Key Concepts of Support Vector Machine

- **Hyperplane:** *A decision boundary separating different classes in feature space and is represented by the equation $wx + b = 0$ in linear classification.*
- **Support Vectors:** *The closest data points to the hyperplane, crucial for determining the hyperplane and margin in SVM.*
- **Margin:** *The distance between the hyperplane and the support vectors. SVM aims to maximize this margin for better classification performance.*
- **Kernel:** *A function that maps data to a higher-dimensional space enabling SVM to handle non-linearly separable data.*
- **Hard Margin:** *A maximum-margin hyperplane that perfectly separates the data without misclassifications.*

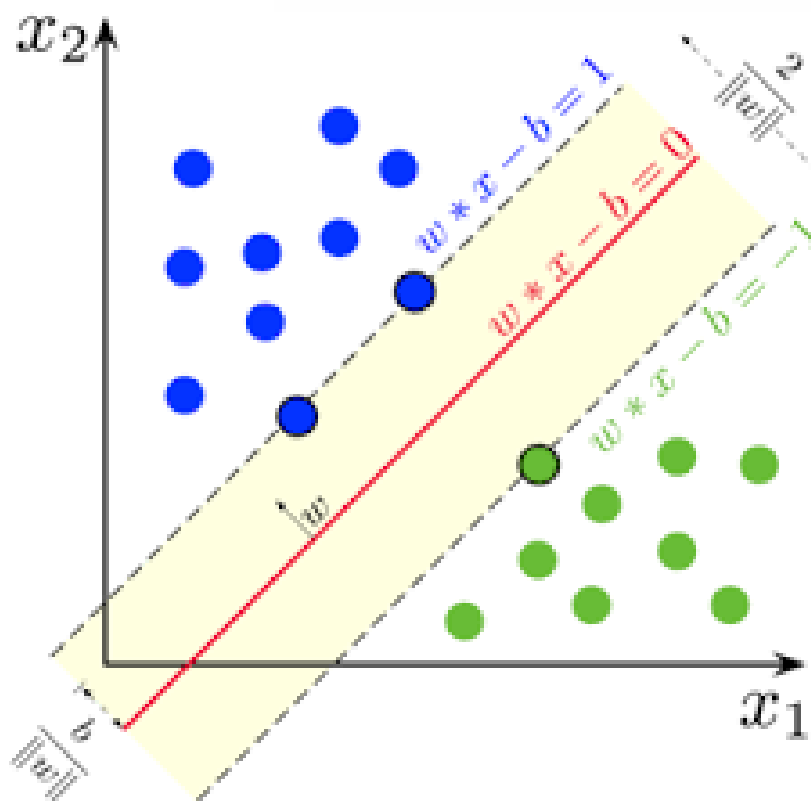
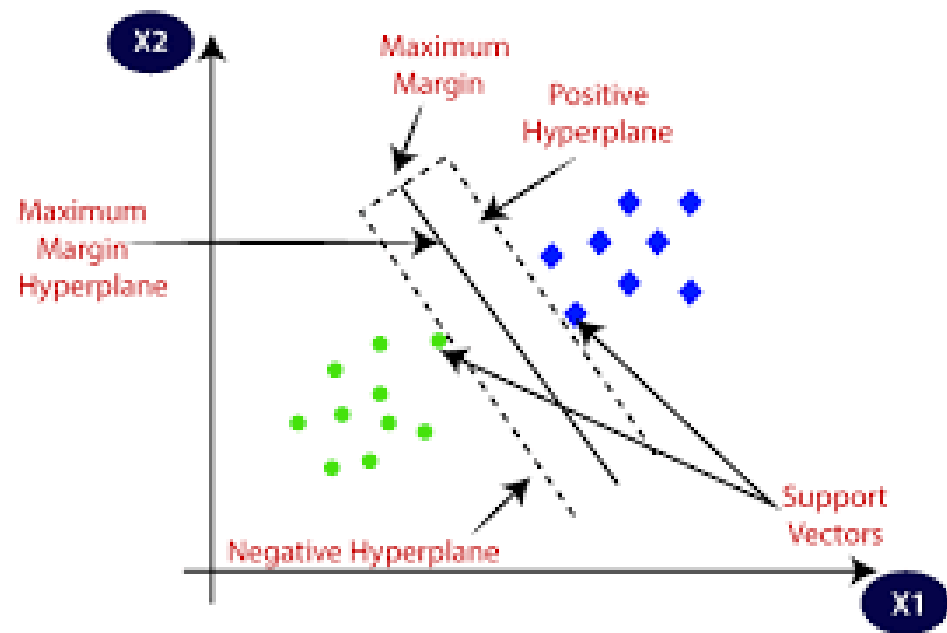
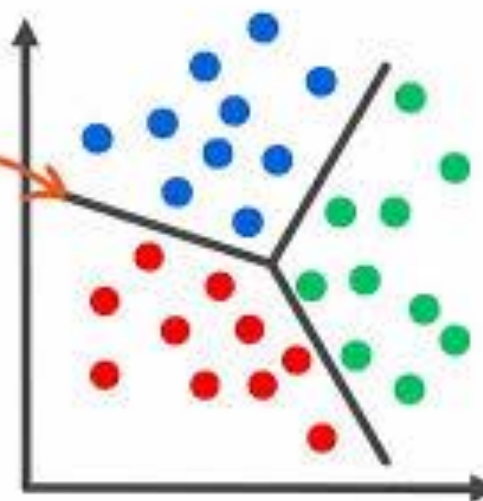
- **Soft Margin:** Allows some misclassifications by introducing slack variables, balancing margin maximization and misclassification penalties *when data is not perfectly separable*.
- **C:** *A regularization term* balancing margin maximization and misclassification penalties. A higher C value forces stricter penalty for misclassifications.
- **Hinge Loss:** A loss function penalizing misclassified points or margin violations and is combined with regularization in SVM.
- **Dual Problem:** Involves solving for *Lagrange multipliers* associated with support vectors, facilitating the *kernel trick and efficient computation*.

Margin (gap between decision boundary and hyperplanes)



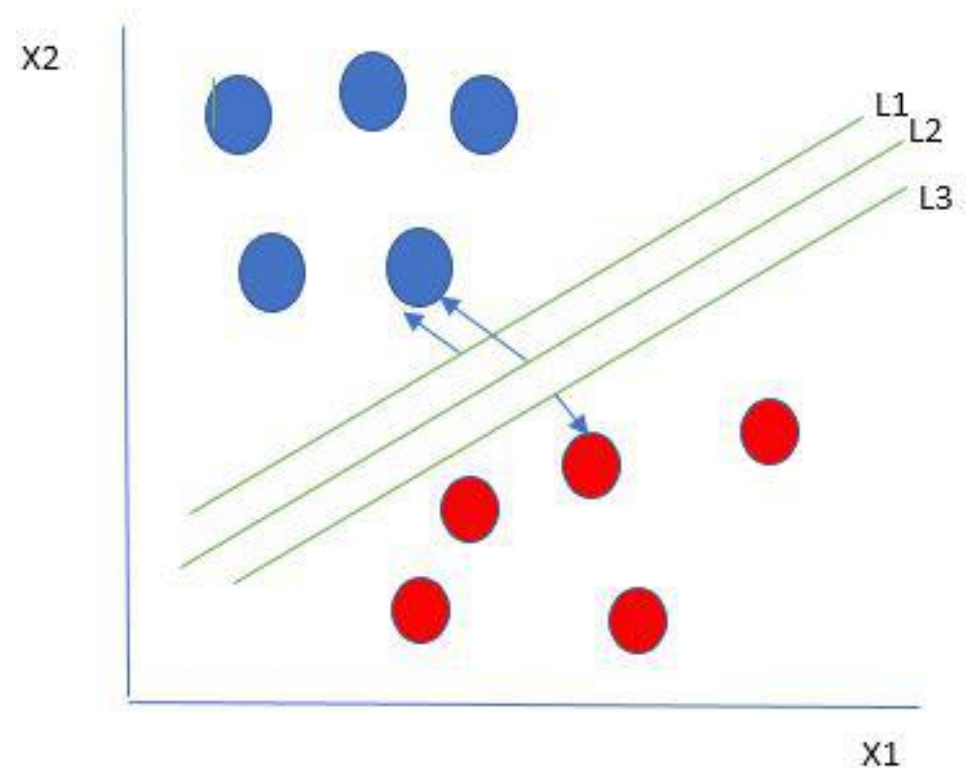
Support Vector
Machines (SVM)

Hyperplanes that Best
Separates Different Classes

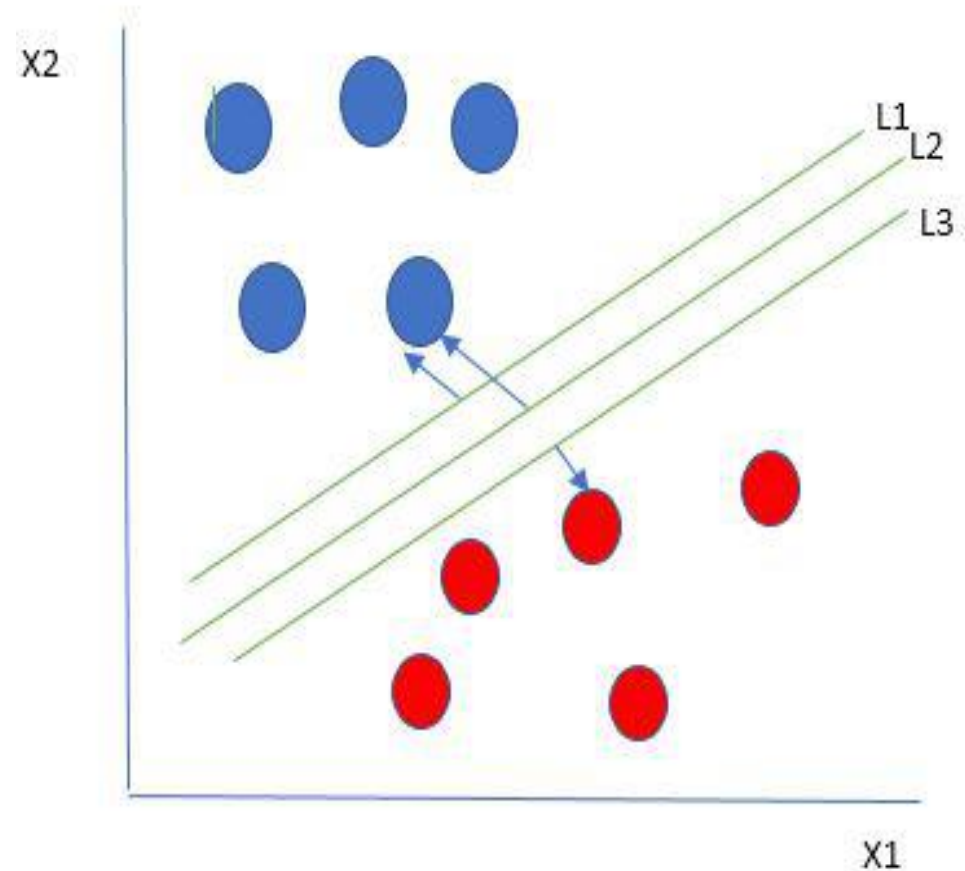


How does Support Vector Machine Algorithm Work?

- The key idea behind the SVM algorithm is **to find the hyperplane that best separates two classes by maximizing the margin between them.**
- This margin is the **distance from the hyperplane to the nearest data points (support vectors) on each side.**

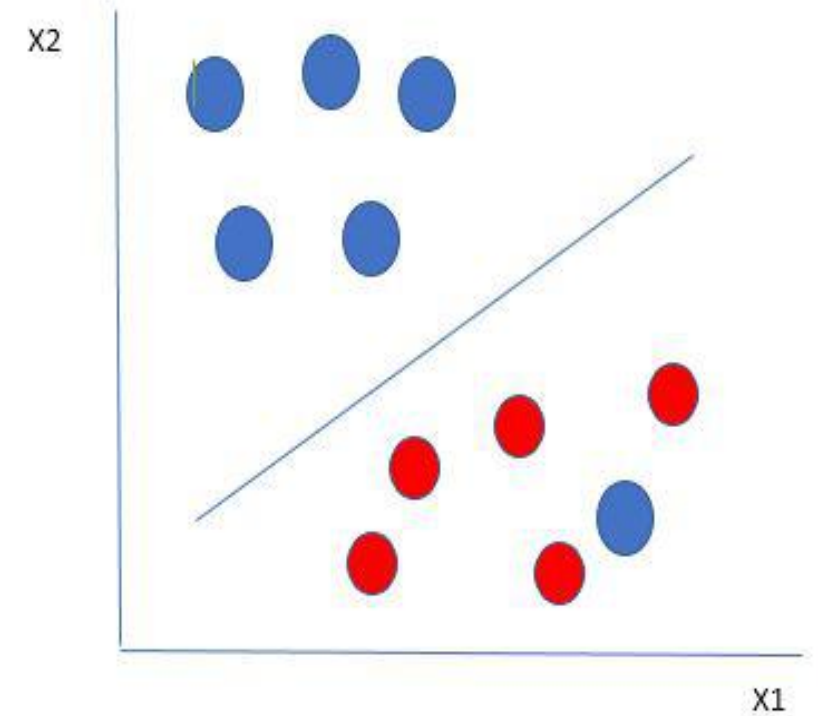


- *The best hyperplane also known as the "hard margin" is the one that maximizes the distance between the hyperplane and the nearest data points from both classes.*
- This ensures a clear separation between the classes. So from the below figure, we choose L2 as hard margin.



How does SVM classify the data?

- The blue ball in the boundary of red ones *is an outlier of blue balls.*
- The SVM algorithm has the characteristics *to ignore the outlier* and *finds the best hyperplane that maximizes the margin.*
- SVM is robust to outliers.



Soft Margin & Objective Function

A soft margin allows for some misclassifications or violations of the margin to improve generalization. The SVM optimizes the following equation to balance margin maximization and penalty minimization:

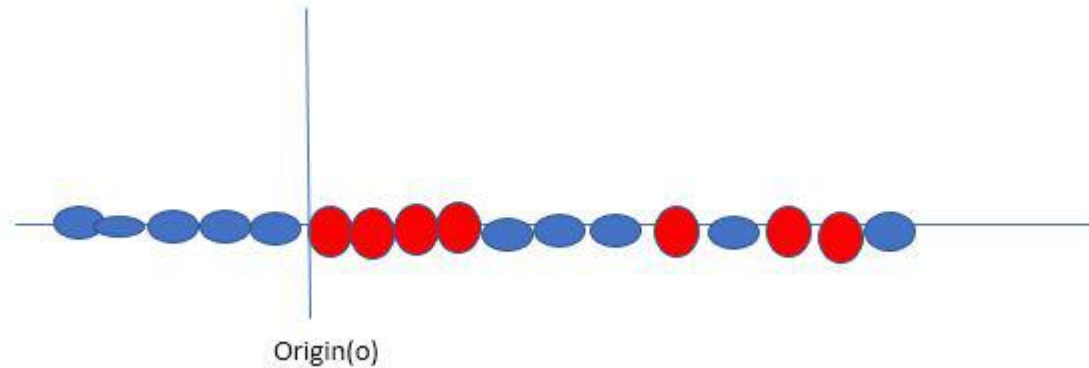
$$\text{Objective Function} = \left(\frac{1}{\text{margin}} \right) + \lambda \sum \text{penalty}$$

The penalty used for violations is often hinge loss which has the following behavior:

- If a data point is correctly classified and within the margin there is no penalty (loss = 0).
- If a point is incorrectly classified or violates the margin the hinge loss increases proportionally to the distance of the violation.

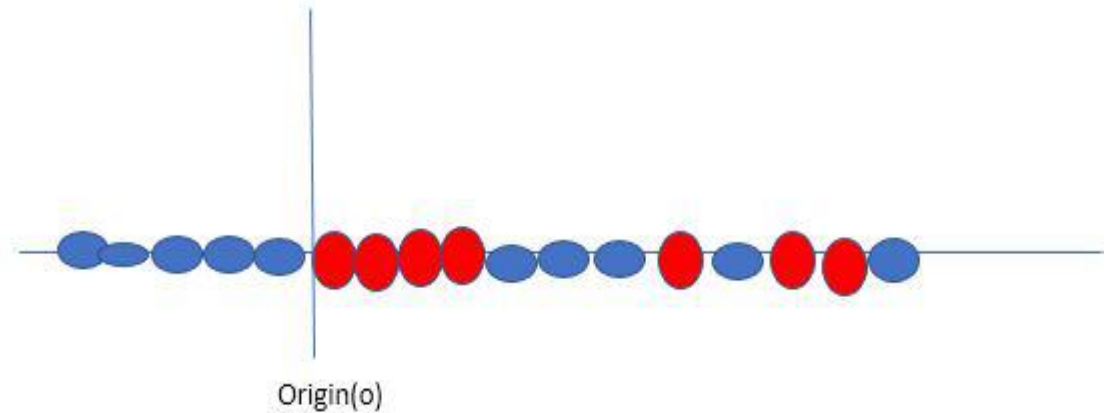
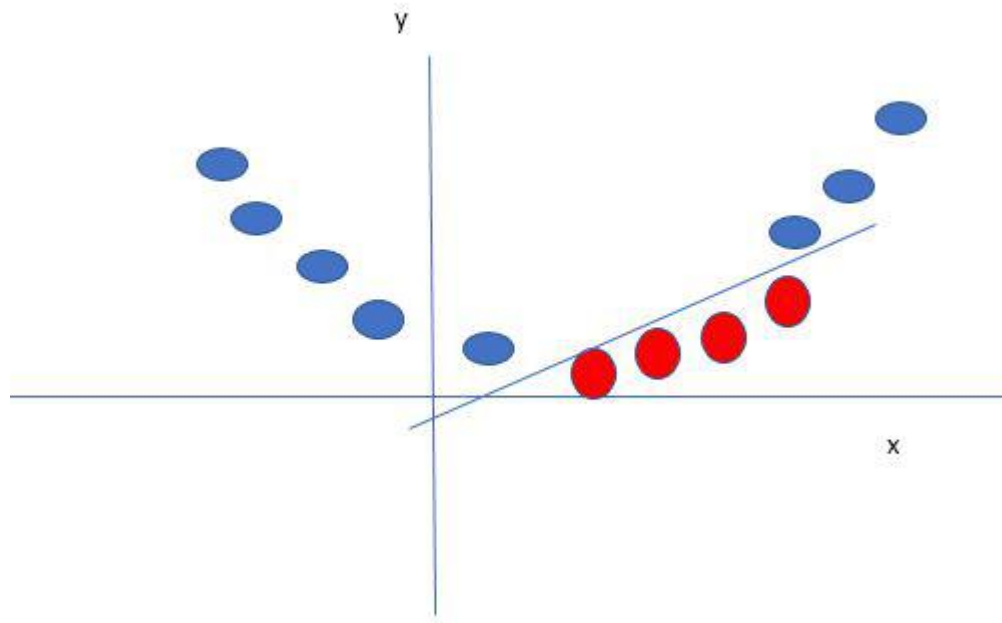
What if data is not linearly separable?

- When data is not linearly separable i.e it can't be divided by a straight line, SVM uses a technique called [kernels](#) to map the data into a higher-dimensional space where it becomes separable.
- This transformation helps SVM find a decision boundary even for non-linear data.



- *A kernel is a function that maps data points into a higher-dimensional space* without explicitly computing the coordinates in that space.
- This allows SVM to work efficiently with non-linear data *by implicitly performing the mapping*. For example consider data points that are not linearly separable.
- By applying a kernel function SVM transforms the data points into a higher-dimensional space where they become linearly separable.
- **Linear Kernel:** For linear separability.
- **Polynomial Kernel:** Maps data into a polynomial space.
- **Radial Basis Function (RBF) Kernel:** Transforms data into a space based on distances between data points.

In this case the new variable y is created as a function of distance from the origin.



Computation of SVM

Consider a binary classification problem with two classes, labeled as +1 and -1. We have a training dataset consisting of input feature vectors X and their corresponding class labels Y . The equation for the linear hyperplane can be written as:

$$w^T x + b = 0$$

Where:

- w is the normal vector to the hyperplane (the direction perpendicular to it).
- b is the offset or bias term representing the distance of the hyperplane from the origin along the normal vector w .

Distance from a Data Point to the Hyperplane

The distance between a data point x_i and the decision boundary can be calculated as:

$$d_i = \frac{w^T x_i + b}{||w||}$$

where $||w||$ represents the Euclidean norm of the weight vector w .

Linear SVM Classifier

Distance from a Data Point to the Hyperplane:

$$\hat{y} = \begin{cases} 1 & : w^T x + b \geq 0 \\ 0 & : w^T x + b < 0 \end{cases}$$

Where $\hat{y}$ is the predicted label of a data point.

Types of Support Vector Machine

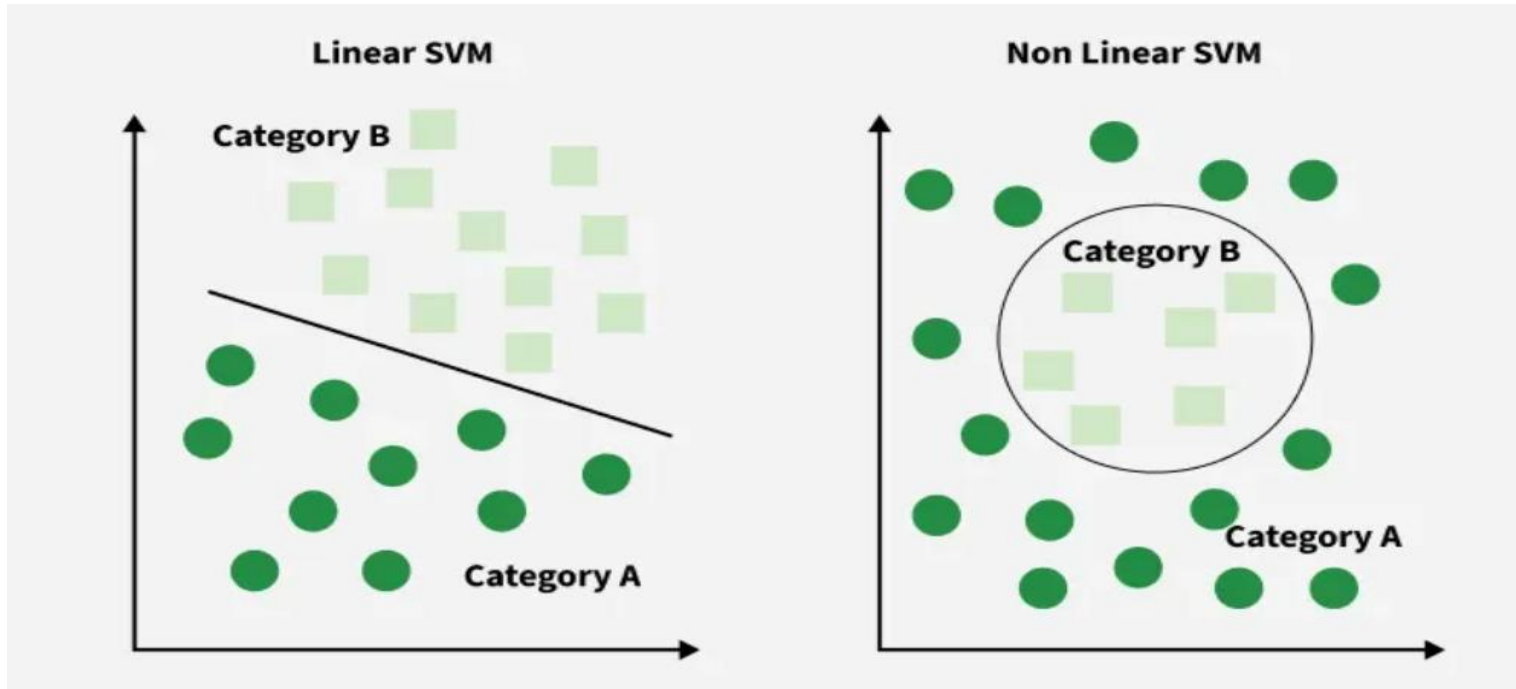
- **Linear SVM:** Linear SVMs use a linear decision boundary to separate the data points of different classes.
- When the data can be precisely linearly separated, linear SVMs are very suitable.
- This means that a single straight line (in 2D) or a hyperplane (in higher dimensions) can entirely divide the data points into their respective classes.
- A hyperplane that maximizes the margin between the classes is the decision boundary.

Non-Linear SVM

- It can be used *to classify data when it cannot be separated into two classes* by a straight line (in the case of 2D).
- By using kernel functions, nonlinear SVMs can handle nonlinearly separable data.
- The original input data is transformed by these kernel functions into a higher-dimensional feature space where the data points can be linearly separated.
- A linear SVM is used to locate a nonlinear decision boundary in this modified space.

Non-Linear SVM

- The standard (linear) SVM can only classify data that is linearly separable, meaning a straight line can separate the classes (in 2D) or a hyperplane (in higher dimensions).
- Non-Linear SVM extends SVM to handle complex, non-linearly separable data using kernels.



- For example, one group of points might surround another group in a circle.
- A simple SVM won't work well here because it only draws straight lines.
- Non-linear SVM is needed because it can draw curved lines to separate such data properly.
- This helps the model make better predictions when the data has complex shapes or patterns.
- Non-Linear SVM uses kernels to work in higher dimensions where data can become linearly separable.

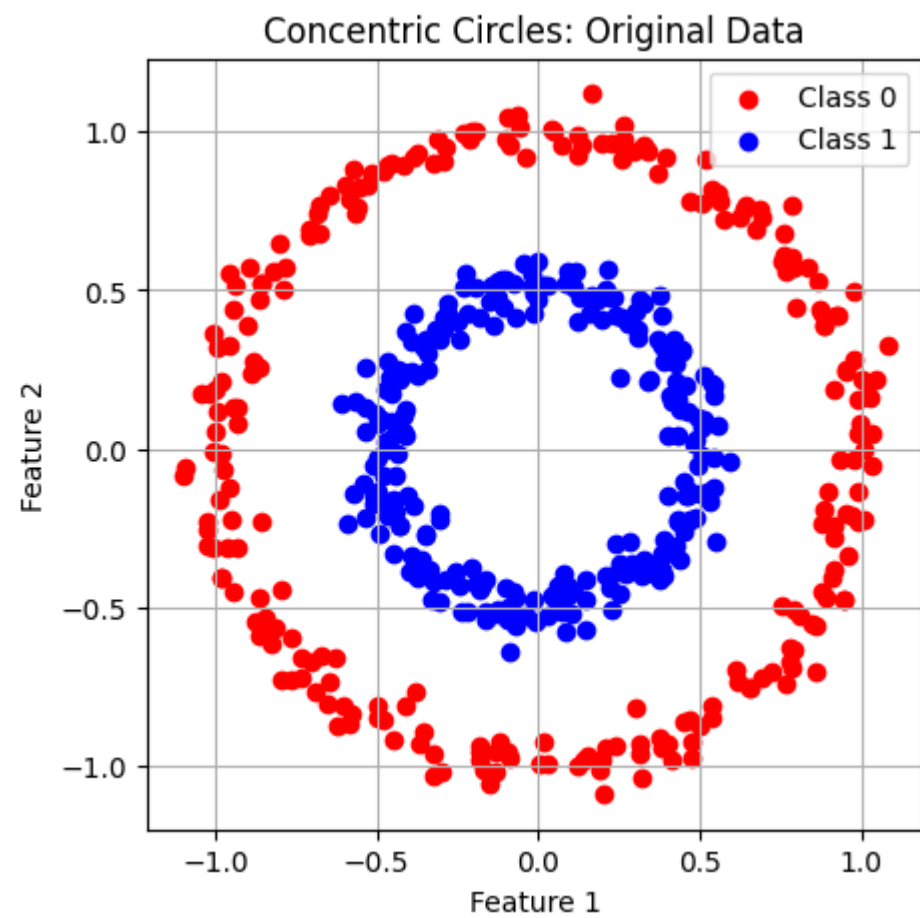
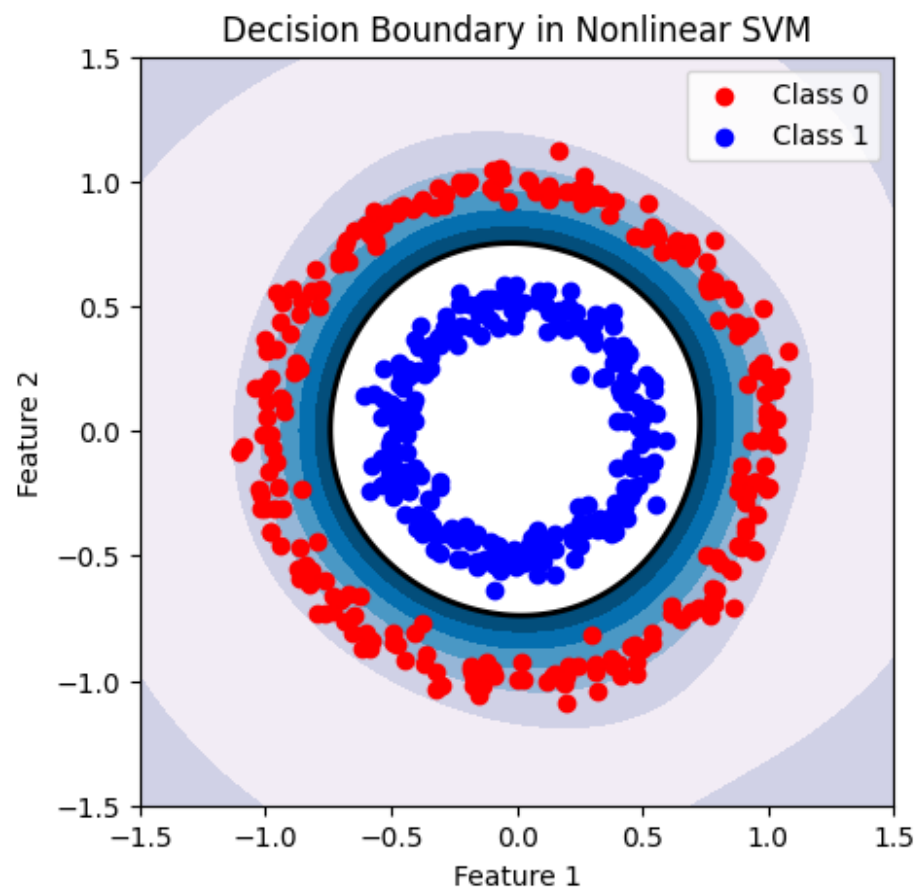
What is Kernel?

- Instead of explicitly computing the transformation the kernel computes the **dot product of data points in the higher-dimensional space directly that helps a model find patterns in complex data and transforming** the data into a higher-dimensional space where it becomes easier to separate different classes or detect relationships.
- ***For example,*** suppose we have data points shaped like two concentric circles: one circle represents one class and the other circle represents another class.
- If we try to separate these classes with a straight line it can't be done because the data is not linearly separable in its current form.

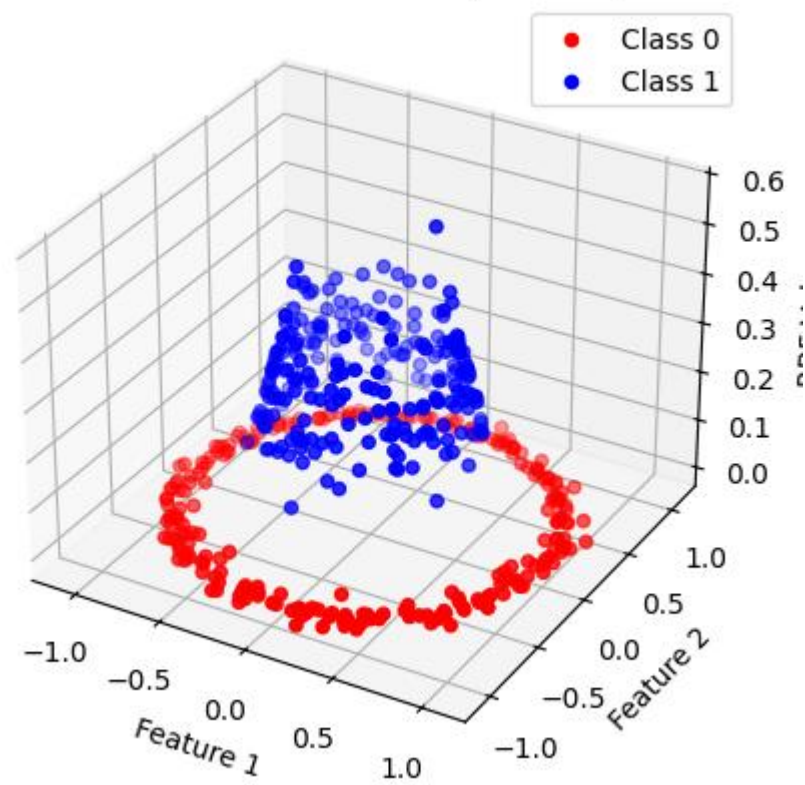
- When we use a kernel function it transforms the original 2D data like the concentric circles into a higher-dimensional space where the data becomes linearly separable.
- In that higher-dimensional space the SVM finds a simple straight-line decision boundary to separate the classes.
- When we bring this straight-line decision boundary back to the original 2D space it no longer looks like a straight line.
- Instead, it appears as a circular boundary that perfectly separates the two classes.
- This happens because the kernel trick allows the SVM to "see" the data in a new way enabling it to draw a boundary that fits the original shape of the data.

Popular kernel functions in SVM

- **Radial Basis Function (RBF):** Captures patterns in data by measuring the distance between points and is ideal for circular or spherical relationships. It is widely used as it creates flexible decision boundary.
- **Linear Kernel:** Works for data that is linearly separable problem without complex transformations.
- **Polynomial Kernel:** Models more complex relationships using polynomial equations.
- **Sigmoid Kernel:** Mimics neural network behavior using sigmoid function and is suitable for specific non-linear problems.



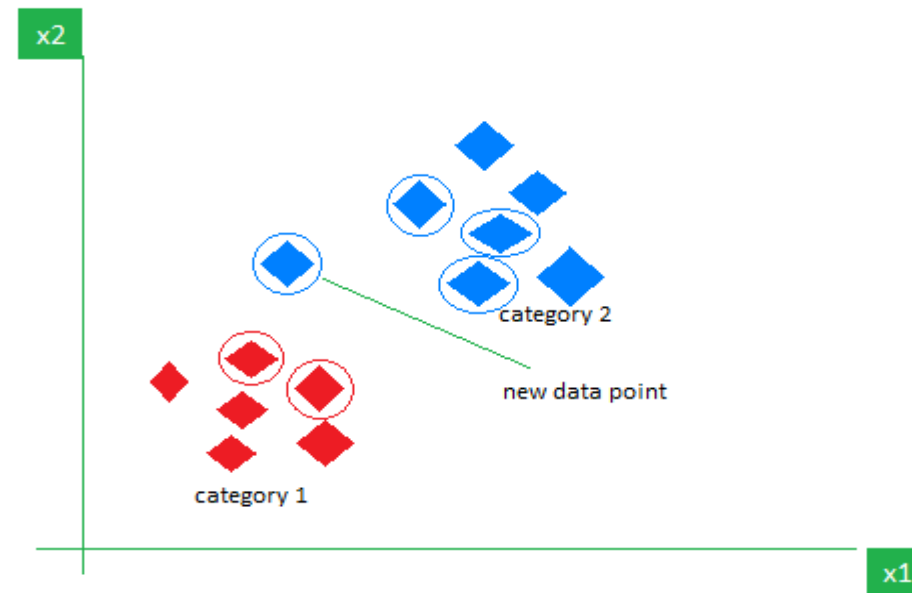
Radial Transformation (3D View)



K-Nearest Neighbor(KNN)

- K-Nearest Neighbors (KNN) is a supervised machine learning algorithm *generally used for classification* but can also be used for *regression tasks*.
- It works by *finding the "k" closest data points (neighbors) to a given input* and makes a predictions based on the *majority class (for classification)* or the *average value (for regression)*.
- Since KNN makes no assumptions about the underlying data distribution it makes it a *non-parametric and instance-based learning method*.

- K-Nearest Neighbors is also called as a *lazy learner algorithm* because it does not learn from the training set immediately instead it *stores the dataset and at the time of classification* it performs an action on the dataset.
- *For example,* consider the following table of data points containing two features:



- The new point is classified as Category 2 because most of its closest neighbors are blue squares. KNN assigns the category based on the *majority of nearby points*. The image shows how KNN predicts the category of a new data point based on its closest neighbours.
- The red diamonds represent Category 1 and the blue squares represent Category 2.
- The new data point checks its closest neighbors (circled points).
- Since the majority of its closest neighbors are blue squares (Category 2) KNN predicts the new data point belongs to Category 2.
- KNN works *by using proximity and majority voting* to make predictions.

What is 'K' in K Nearest Neighbour?

- In the k-Nearest Neighbours algorithm *k is just a number that tells the algorithm how many nearby points or neighbors* to look at when it makes a decision.
- **Example:** Imagine you're deciding which fruit it is based on its shape and size. You compare it to fruits you already know.
- If $k = 3$, the algorithm looks at the 3 closest fruits to the new one.
- If 2 of those 3 fruits are apples and 1 is a banana, the algorithm says the new fruit is an apple because most of its neighbors are apples.

How to choose the value of k for KNN Algorithm?

- The value of k in KNN decides how many neighbors the algorithm looks at when making a prediction.
- *Choosing the right k is important for good results.*
- If the data has lots of noise or outliers, using a larger k can make the predictions more stable.
- But if k is too large the model may become too simple *and miss important patterns and this is called underfitting.*
- So k should be picked carefully based on the data.

Statistical Methods for Selecting k

- **Cross-Validation** is a good way to **find the best value of k** is by using k -fold cross-validation.
- This means dividing the dataset into k parts.
- The model is trained on some of these parts and tested on the remaining ones.
- This process is repeated for each part.
- The ***k value that gives the highest average accuracy during these tests is usually the best one to use.***

- **Elbow Method:** we draw a graph showing the error rate or accuracy for different k values.
- As k increases the error usually drops at first.
- But after a certain point error stops decreasing quickly.
- The point where the *curve changes direction and looks like an "elbow"* is usually the best choice for k .
- **Odd Values for k :** It's a good idea to use an odd number for k especially in classification problems.
- *This helps avoid ties when deciding* which class is the most common among the neighbors.

Distance Metrics Used in KNN Algorithm

1. Euclidean Distance

- Euclidean distance is defined as the straight-line distance between two points in a plane or space.
- You can think of it like the shortest path you would walk if you were to go directly from one point to another.

$$\text{distance}(x, X_i) = \sqrt{\sum_{j=1}^d (x_j - X_{i_j})^2}$$

2. Manhattan Distance

- This is the total distance you would travel if you could only move along horizontal and vertical lines like a grid or city streets.
- It's also called "taxicab distance" because a taxi can only drive along the grid-like streets of a city.

$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

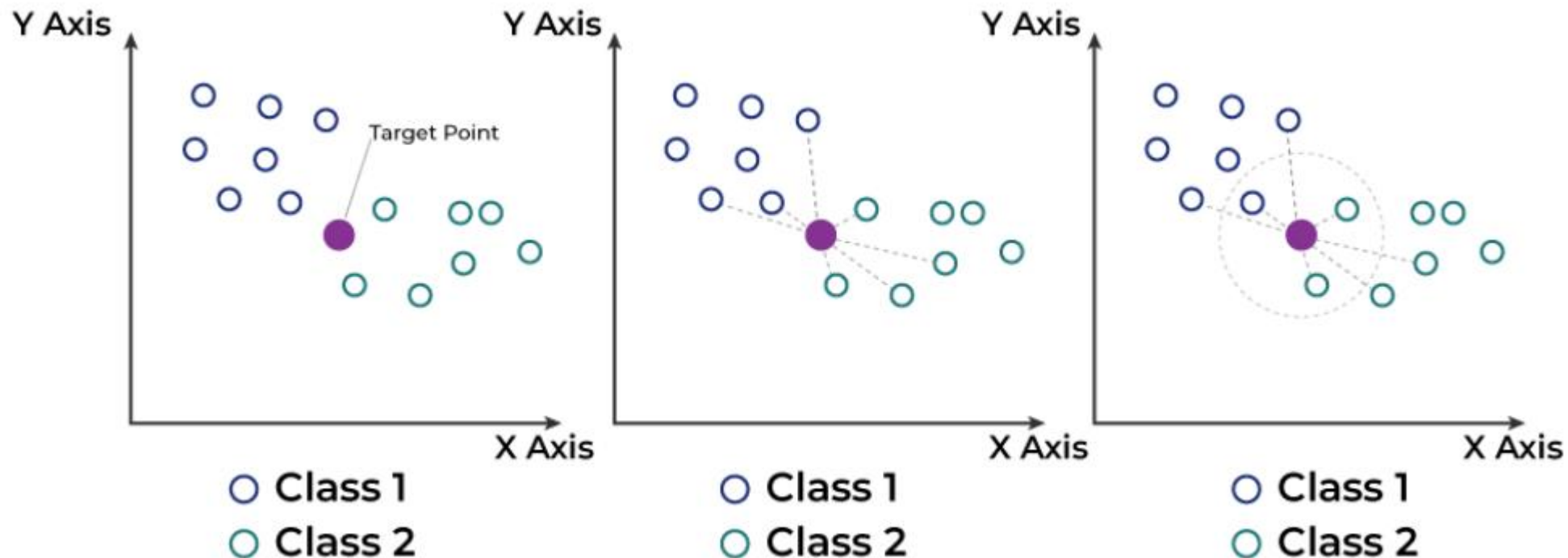
3. Minkowski Distance

- Minkowski distance is like a family of distances, which includes both *Euclidean and Manhattan* distances as special cases.
- From the formula above, when $p=2$, it becomes the same as the Euclidean distance formula and when $p=1$, it turns into the Manhattan distance formula.
- Minkowski distance is essentially a flexible formula that can represent either Euclidean or Manhattan distance depending on the value of p .

$$d(x, y) = \left(\sum_{i=1}^n (x_i - y_i)^p \right)^{\frac{1}{p}}$$

Working of KNN algorithm

- The K-Nearest Neighbors (KNN) algorithm operates on the principle of similarity where it predicts the label or value of a new data point by considering the labels or values of its K nearest neighbors in the training dataset.



Step 1: Selecting the optimal value of K

- K represents the number of nearest neighbors that needs to be considered while making prediction.

Step 2: Calculating distance

- To measure the similarity between target and training data points Euclidean distance is used. Distance is calculated between data points in the dataset and target point.

Step 3: Finding Nearest Neighbors

- The k data points with the smallest distances to the target point are nearest neighbors.

Step 4: Voting for Classification or Taking Average for Regression

- When you want to classify a data point into a category like spam or not spam, the KNN algorithm looks at the K closest points in the dataset. These closest points are called neighbors. The algorithm then looks at which category the neighbors belong to and picks the one that appears the most. This is called majority voting.
- In regression, the algorithm still looks for the K closest points. But instead of voting for a class in classification, it takes the average of the values of those K neighbors. This average is the predicted value for the new point for the algorithm.

K-Nearest Neighbor (KNN) – Numerical Problem

Given the following training dataset:

Instance	x_1	x_2	Output
l_1	7	7	0
l_2	7	4	0
l_3	3	4	1
l_4	1	4	1

Using the **K-Nearest Neighbor (KNN)** algorithm with $k = 3$, classify the **test instance (3, 7)**. Use **Euclidean distance** to determine the nearest neighbors.

Step 1: Compute Euclidean distances. $x_1'=3, x_2'=7, k=3$

$$d = \sqrt{(x_1 - x'_1)^2 + (x_2 - x'_2)^2}$$

For $I_1 (7,7)$:

$$d = \sqrt{(7 - 3)^2 + (7 - 7)^2} = \sqrt{16 + 0} = \sqrt{16} = 4$$

For $I_2 (7,4)$:

$$d = \sqrt{(7 - 3)^2 + (4 - 7)^2} = \sqrt{16 + 9} = \sqrt{25} = 5$$

For $I_3 (3,4)$:

$$d = \sqrt{(3 - 3)^2 + (4 - 7)^2} = \sqrt{0 + 9} = \sqrt{9} = 3$$

For $I_4 (1,4)$:

$$d = \sqrt{(1 - 3)^2 + (4 - 7)^2} = \sqrt{4 + 9} = \sqrt{13} \approx 3.6$$

Step 2: Sort distances and select k=3 nearest neighbors

Instance	Output	Distance
l_3	1	3
l_4	1	3.6
l_1	0	4
l_2	0	5

Top 3 nearest neighbors:

$l_3 \rightarrow$ Output 1

$l_4 \rightarrow$ Output 1

$l_1 \rightarrow$ Output 0

Step 4: Perform majority voting

- Outputs of the 3 nearest neighbors: [1, 1, 0]
- Count of each class:
- Class 1 $\rightarrow$ 2 votes
- Class 0 $\rightarrow$ 1 vote

Solution:

Final Prediction: Class 1

The query instance (3,7) is classified as 1 using KNN with $k = 3$.

Given the following training dataset:

Instance	x_1	x_2	Output
I_1	6	7	0
I_2	7	3	0
I_3	4	4	1
I_4	2	5	1
I_5	1	2	1
I_6	5	8	0
I_7	3	3	1

Using the **K-Nearest Neighbor (KNN)** algorithm with **$k = 3$** , classify the **test instance (3, 6)**. Use **Euclidean distance** to determine the nearest neighbors.

Step 1: Compute Euclidean distances

$$\text{Distance} = \sqrt{(x_1 - x'_1)^2 + (x_2 - x'_2)^2}$$

$l_1: (6, 7)$

$$\sqrt{(6 - 3)^2 + (7 - 6)^2} = \sqrt{9 + 1} = \sqrt{10} \approx 3.16$$

$l_2: (7, 3)$

$$\sqrt{(7 - 3)^2 + (3 - 6)^2} = \sqrt{16 + 9} = \sqrt{25} = 5$$

$l_3: (4, 4)$

$$\sqrt{(4 - 3)^2 + (4 - 6)^2} = \sqrt{1 + 4} = \sqrt{5} \approx 2.24$$

$l_4: (2, 5)$

$$\sqrt{(2 - 3)^2 + (5 - 6)^2} = \sqrt{1 + 1} = \sqrt{2} \approx 1.41$$

$l_5: (1, 2)$

$$\sqrt{(1 - 3)^2 + (2 - 6)^2} = \sqrt{4 + 16} = \sqrt{20} \approx 4.47$$

$l_6: (5, 8)$

$$\sqrt{(5 - 3)^2 + (8 - 6)^2} = \sqrt{4 + 4} = \sqrt{8} \approx 2.83$$

$l_7: (3, 3)$

$$\sqrt{(3 - 3)^2 + (3 - 6)^2} = \sqrt{0 + 9} = \sqrt{9} = 3$$

- **Step 2: Sort by distance**

Instance	Output	Distance
l_4	1	1.41
l_3	1	2.24
l_6	0	2.83
l_7	1	3.00
l_1	0	3.16
l_5	1	4.47
l_2	0	5.00

- **Step 3: Choose top 3 nearest neighbors ($k = 3$)**

- Top 3:

- $I_4 \rightarrow \text{Output} = 1$

- $I_3 \rightarrow \text{Output} = 1$

- $I_6 \rightarrow \text{Output} = 0$

- **Step 4: Majority Voting**

- Votes:

- Class 1 $\rightarrow$ 2 votes (I_4, I_3)

- Class 0 $\rightarrow$ 1 vote (I_6)

Solution: Test instance (3, 6) is classified as **Class 1** using KNN with $k = 3$.

Overfitting

- Overfitting is a situation where a machine learning model learns the training data too well, *including its noise and minor details*. As a result, *it performs very well on the training data but poorly on new, unseen data*.
- High accuracy on training data
- Low accuracy on testing data
- Model becomes too complex and memorizes the data rather than generalizing from it.

Example: A student who memorizes every question and answer from past exams may fail to answer new types of questions in the actual exam.

Underfitting

- Underfitting occurs when a machine learning model is too ***simple to capture the underlying patterns in the data***, resulting in poor performance on both training and testing data.
- Low accuracy on training data
- Low accuracy on testing data
- The model cannot learn enough from the data.

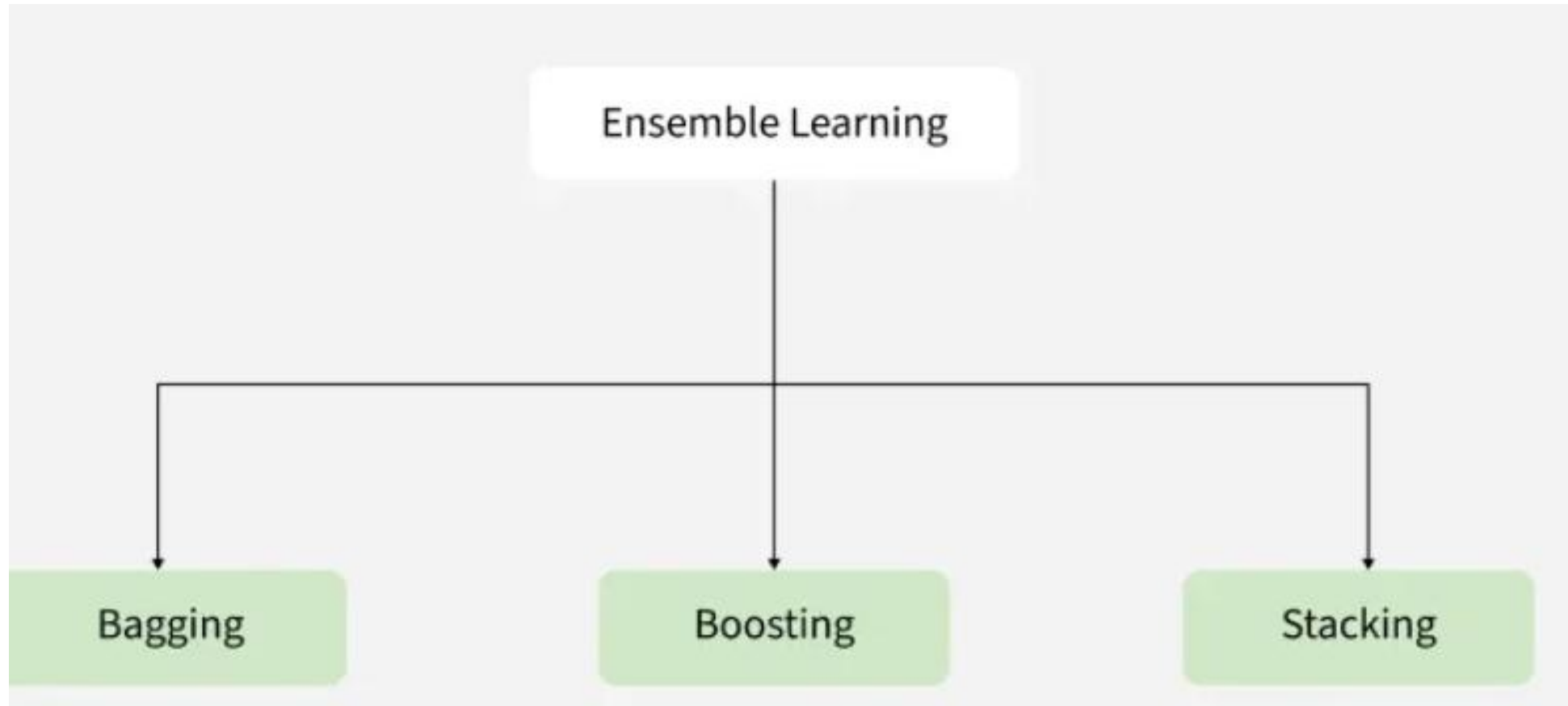
Example: A student who barely studies the subject cannot answer either old or new questions well.

MODULE III

Ensemble Learning Model

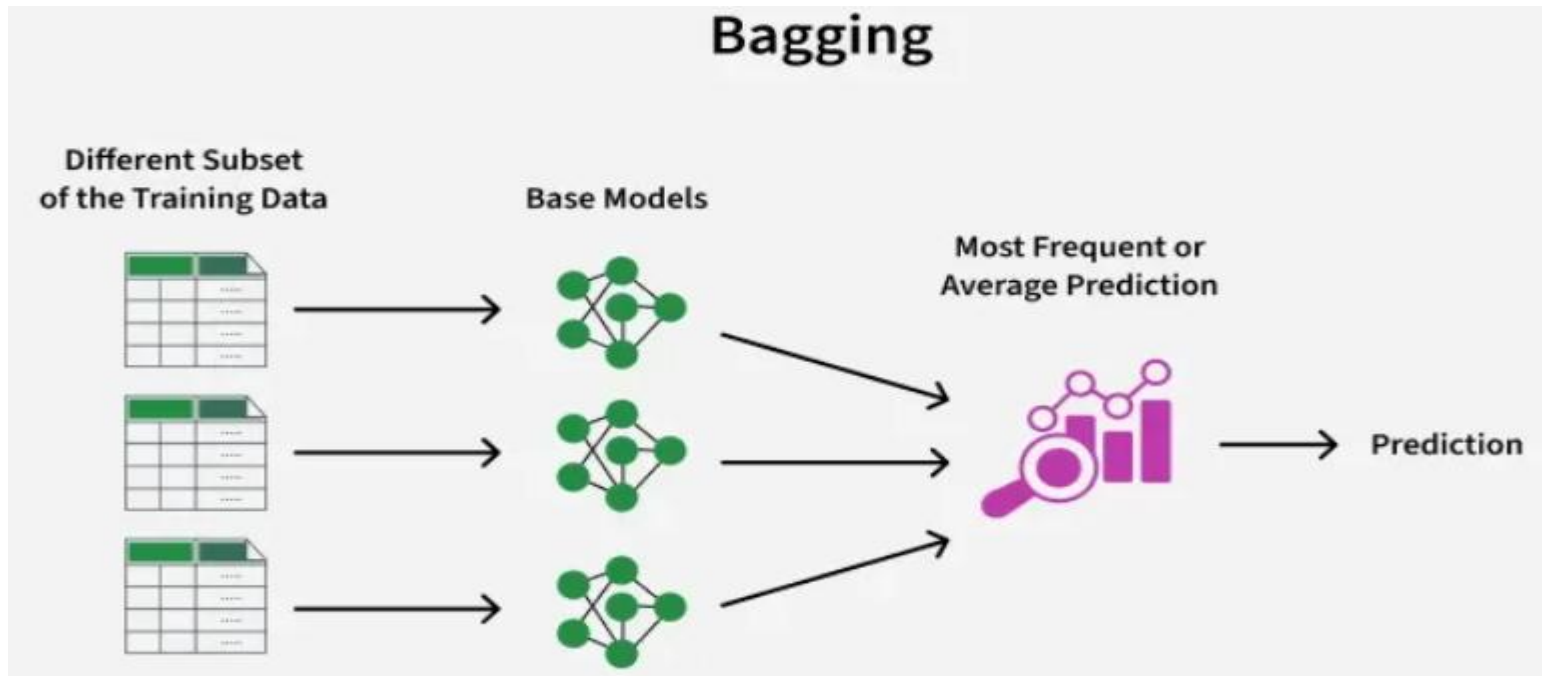
- Ensemble learning is a method *where we use many small models instead of just one.*
- Each of these models may not be very strong on its own, but when we put their results together, *we get a better and more accurate answer.*
- *It's like asking a group of people for advice instead of just one person—each one might be a little wrong, but together, they usually give a better answer.*

Types of Ensembles Learning in Machine Learning



1. Bagging (Bootstrap Aggregating):

- Models are trained independently on *different random subsets of the training data*.
- Their results are then combined, usually by *averaging (for regression) or voting (for classification)*.
- This helps *reduce variance and prevents overfitting*.



Bagging Algorithm

- It can be used for *both regression and classification tasks*.

1. Bootstrap Sampling:

- Divides the original training data into 'N' subsets and randomly selects a subset with replacement in some rows from other subsets.
- This step ensures that the base models are trained on diverse subsets of the data and there is no class imbalance.

2. Base Model Training:

- For each bootstrapped sample we train a base model independently on that subset of data.
- These weak models are trained in parallel to increase computational efficiency and reduce time consumption.
- We can use different base learners i.e. different ML models as base learners to bring variety and robustness.

3. Prediction Aggregation:

- To make a prediction on testing data combine the predictions of all base models.
- For *classification tasks it can include majority voting* or weighted majority while for *regression it involves averaging the predictions*.

4. Out-of-Bag (OOB) Evaluation:

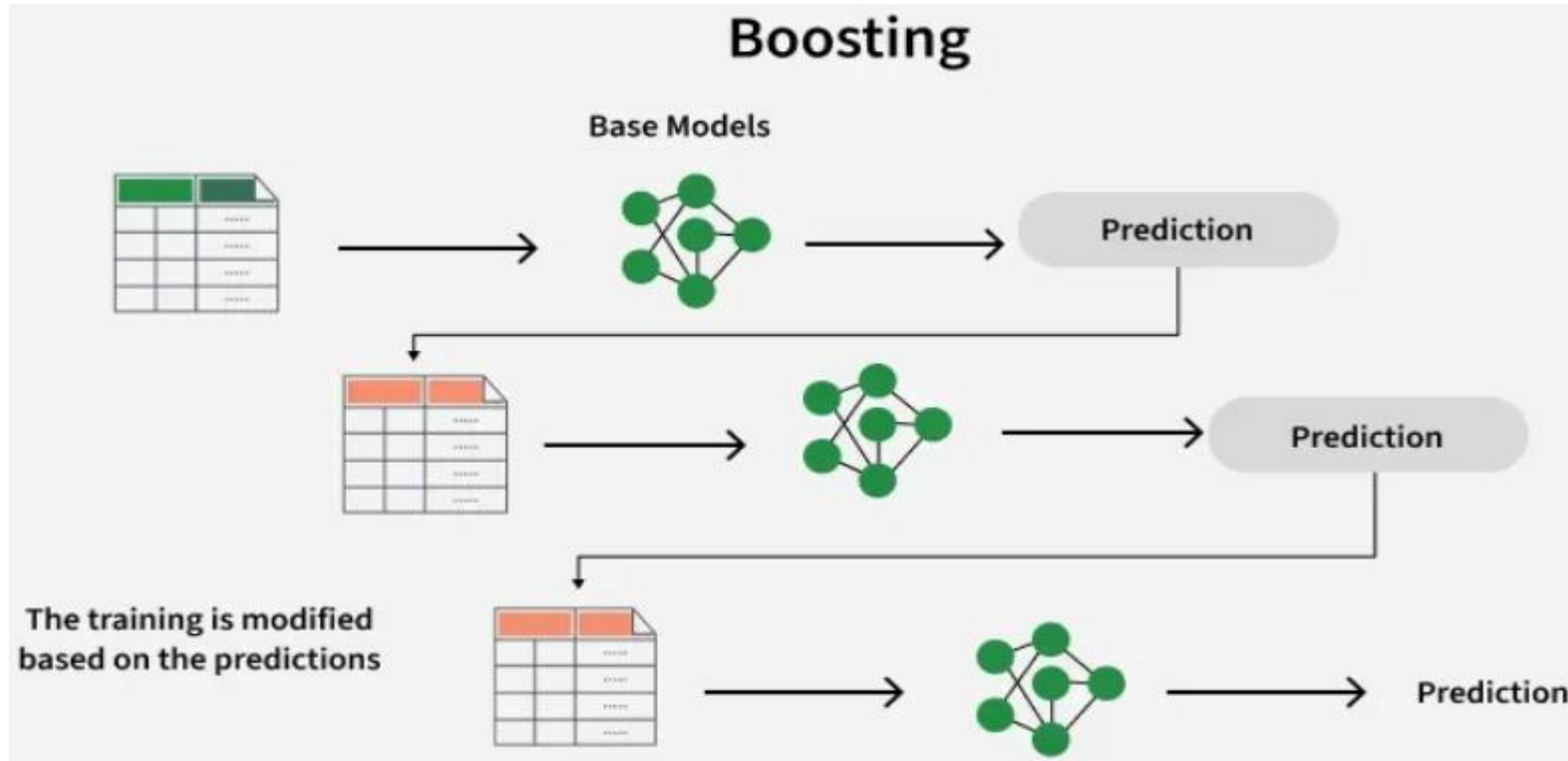
- Some samples are excluded from the training subset of particular base models during the bootstrapping method.
- These “out-of-bag” samples can be used to estimate the model’s performance without the need for cross-validation.

5. Final Prediction:

After aggregating the predictions from all the base models, Bagging produces a final prediction for each instance.

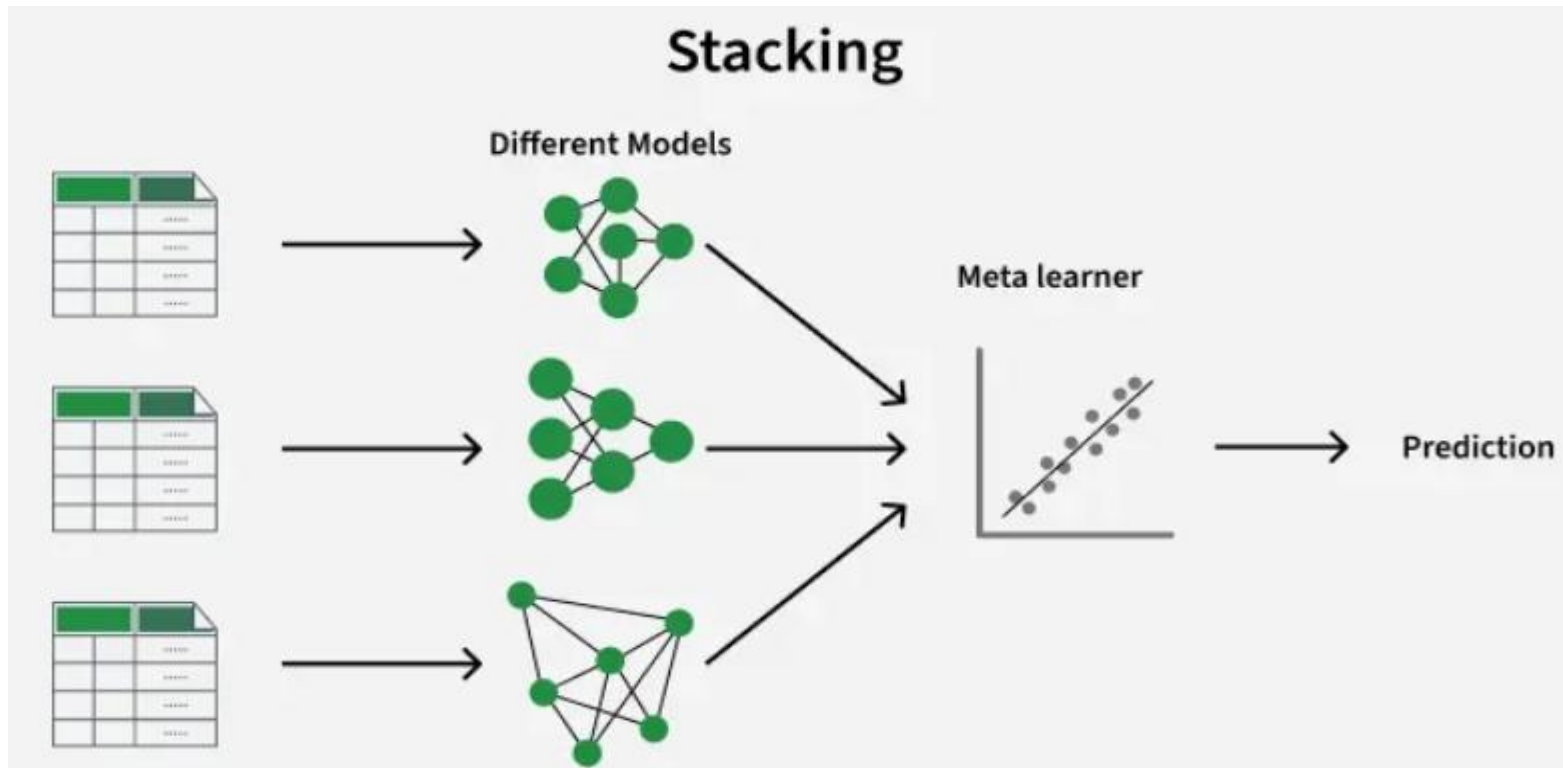
2. Boosting:

- Models are trained one after another.
- Each new model focuses on fixing the errors made by the previous ones.
- The final prediction is a weighted combination of all models, which helps reduce bias and improve accuracy.



3. Stacking (Stacked Generalization):

- Multiple different models (often of different types) are trained, and their *predictions are used as inputs to a final model, called a meta-model*.
- The meta-model learns how to best combine the predictions of the base models, aiming for better performance than any individual model.



- Perform **K-Means Clustering** with $K=2$ and **Euclidean distance** for the given dataset. Determine the final clusters and their centroid coordinates, showing all calculation steps.

Point	X	Y
Q1	1	2
Q2	1	4
Q3	3	3
Q4	5	8
Q5	6	8
Q6	7	7

THANK YOU!!!